

PV - repetition - no_frac - Serial position analysis

```
# do this in calling R script
# library(ggplot2)
# library(fmsb) # for TestModels
# library(lme4)
# library(kableExtra)
# library(MASS) # for dropterm
# library(tidyverse)
# library(dominanceanalysis)
# library(cowplot) # for multiple plots in one figure
# library(formatters) # to wrap strings for plot titles
# library(pals) # for continuous color palettes

# load needed functions
# source(paste0(RootDir, "/src/function_library/sp_functions.R"))
# do this in the calling R script

# for testing
# CurPat <- "GM"
# CurTask <- "repetition"
# MinLength <- 4
# MaxLength <- 10
# BestModelIndexL1 <- 1
# BestModelIndexL2 <- 1
# BestModelIndexL3 <- 1
# ReportTitle <- paste0(CurPat, " - ", CurTask, " - ", RunLabel, " - Serial position analysis")
# RandomSamples <- 10
# DoSimulations <- FALSE
# RemoveFracPres <- TRUE

CurPat <- params$CurPat
CurTask <- params$CurTask
MinLength <- params$MinLength
MaxLength <- params$MaxLength
BestModelIndexL1 <- params$BestModelIndexL1
BestModelIndexL2 <- params$BestModelIndexL2
BestModelIndexL3 <- params$BestModelIndexL3
ReportTitle <- params$ReportTitle
RandomSamples <- params$RandomSamples
DoSimulations <- params$DoSimulations
RemoveFracPres <- params$RemoveFracPres

# done in calling script
# print(paste0("Using root dir: ", RootDir))
# RunDir <- paste0(paste0(RootDir, "/output/"), RunLabel))
# if(!dir.exists(RunDir)){
#   dir.create(RunDir, showWarnings = TRUE)
#   print(paste0("created run directory: ", RunDir))
# }
```

```

# # create patient directory if it doesn't already exist
# PatientDir <- paste0(RootDir, "/output/", RunLabel, "/", CurPat)
# if(!dir.exists(PatientDir)){
#   dir.create(PatientDir, showWarnings = TRUE)
#   print(paste0("created participant directory: ", PatientDir))
# }
# TablesDir <- paste0(RootDir, "/output/", RunLabel, "/", CurPat, "/tables/")
# if(!dir.exists(TablesDir)){
#   dir.create(TablesDir, showWarnings = TRUE)
#   print(paste0("created tables directory: ", TablesDir))
# }
# FigDir <- paste0(RootDir, "/output/", RunLabel, "/", CurPat, "/fig/")
# if(!dir.exists(FigDir)){
#   dir.create(FigDir, showWarnings = TRUE)
#   print(paste0("created figures directory: ", FigDir))
# }
# ReportsDir <- paste0(RootDir, "/output/", RunLabel, "/", CurPat, "/reports/")
# if(!dir.exists(ReportsDir)){
#   dir.create(ReportsDir, showWarnings = TRUE)
#   print(paste0("created reports directory: ", ReportsDir))
# }
results_report_DF <- data.frame(Description=character(), ParameterValue=double())

ModelDatFilename<-paste0(RootDir, "/data/", CurPat, "/", CurPat, "_", CurTask, "_posdat.csv")
if(!file.exists(ModelDatFilename)){
  print("**ERROR** Input data file not found")
  print(sprintf("\tfile: ", ModelDatFilename))
  print(sprintf("\troot dir: ", RootDir))
}
PosDat<-read.csv(ModelDatFilename)

# may already be done in datafile
# if(RemoveFinalPosition){
#   PosDat <- PosDat[PosDat$pos < PosDat$stimlen,] # remove final position
# }
PosDat <- PosDat[PosDat$stimlen >= MinLength,] # include only lengths with sufficient N
PosDat <- PosDat[PosDat$stimlen <= MaxLength,]
# include only correct and nonword errors (not no response, word or w+nw errors)
PosDat <- PosDat[(PosDat$err_type == "correct") | (PosDat$err_type == "nonword"), ]
PosDat <- PosDat[(PosDat$pos) != (PosDat$stimlen), ]
# make sure final positions have been removed
PosDat$stim_number <- NumberStimuli(PosDat)
PosDat$raw_log_freq <- PosDat$log_freq
PosDat$log_freq <- PosDat$log_freq - mean(PosDat$log_freq) # centre frequency
OrigDataLen <- nrow(PosDat)
print(sprintf ("Original data has %d values", OrigDataLen))

## [1] "Original data has 3949 values"

if(RemoveFracPres){ # eliminate fractional preserved values?
  PosDat <- PosDat[(PosDat$preserved == 0) | (PosDat$preserved == 1),]
  DataLen<- nrow(PosDat)
  print(sprintf("Data without fractional preserved values has %d values", DataLen))
  print(sprintf("%d values eliminated.", OrigDataLen - DataLen))
}

```

```
## [1] "Data without fractional preserved values has 3823 values"
## [1] "126 values eliminated."
```

```
PosDat$CumPres <- CalcCumPres(PosDat)
PosDat$CumErr <- CalcCumErrFromPreserved(PosDat)
```

```
# plot distribution of complex onsets and codas
# across serial position in the stimuli -- do complexities concentrate
# on one part of the serial position curve?
# syll_component = 1 is coda
# syll_component = S is satellite
```

```
# get counts of syllable components in different serial positions
```

```
syll_comp_dist_N <- PosDat %>% mutate(pos_factor = as.factor(pos), syll_component_factor = as.factor(syll_component))
```

```
syll_comp_dist_perc <- syll_comp_dist_N %>% ungroup() %>%
  mutate(across(O:S, ~ . / total))
```

```
kable(syll_comp_dist_N)
```

pos_factor	O	P	V	1	S	total
1	497	33	117	NA	NA	647
2	58	NA	387	88	98	631
3	276	NA	152	192	12	632
4	271	NA	198	65	33	567
5	207	NA	185	59	35	486
6	177	1	114	63	21	376
7	152	NA	80	24	14	270
8	75	NA	44	22	3	144
9	63	NA	1	NA	6	70

```
kable(syll_comp_dist_perc)
```

pos_factor	O	P	V	1	S	total
1	0.7681607	0.0510046	0.1808346	NA	NA	647
2	0.0919176	NA	0.6133122	0.1394612	0.1553090	631
3	0.4367089	NA	0.2405063	0.3037975	0.0189873	632
4	0.4779541	NA	0.3492063	0.1146384	0.0582011	567
5	0.4259259	NA	0.3806584	0.1213992	0.0720165	486
6	0.4707447	0.0026596	0.3031915	0.1675532	0.0558511	376
7	0.5629630	NA	0.2962963	0.0888889	0.0518519	270
8	0.5208333	NA	0.3055556	0.1527778	0.0208333	144
9	0.9000000	NA	0.0142857	NA	0.0857143	70

```
# get percentages only from summary table
```

```
syll_comp_perc <- syll_comp_dist_perc[,2:(ncol(syll_comp_dist_perc)-1)]
```

```
syll_comp_perc$pos <- as.integer(as.character(syll_comp_dist_perc$pos_factor))
```

```
syll_comp_perc <- syll_comp_perc %>% rename("Coda" = "1", "Satellite" = "S")
```

```
# pivot to long format for plotting
```

```
syll_comp_plot_data <- syll_comp_perc %>% pivot_longer(cols=seq(1:(ncol(syll_comp_perc)-1)),
  names_to="syll_component", values_to="percent") %>% filter(syll_component %in% c("Coda", "Satellite"))
```

```
# plot satellite and coda occurrences across position

syll_comp_plot <- ggplot(syll_comp_plot_data,
                        aes(x=pos,y=percent,group=syll_component,
                           color=syll_component,
                           linetype = syll_component,
                           shape = syll_component)) +
  geom_line() + geom_point() + scale_color_manual(name="Syllable component", values = palette_values) +
syll_comp_plot <- syll_comp_plot + scale_y_continuous(name="Percent of segment types") + scale_linetype
  scale_shape_discrete(name="Syllable component")
ggsave(paste0(FigDir, CurPat, "_", RunLabel, "_", CurTask, "_pos_of_syll_complexities_in_stimuli.png"), plot=sy
```

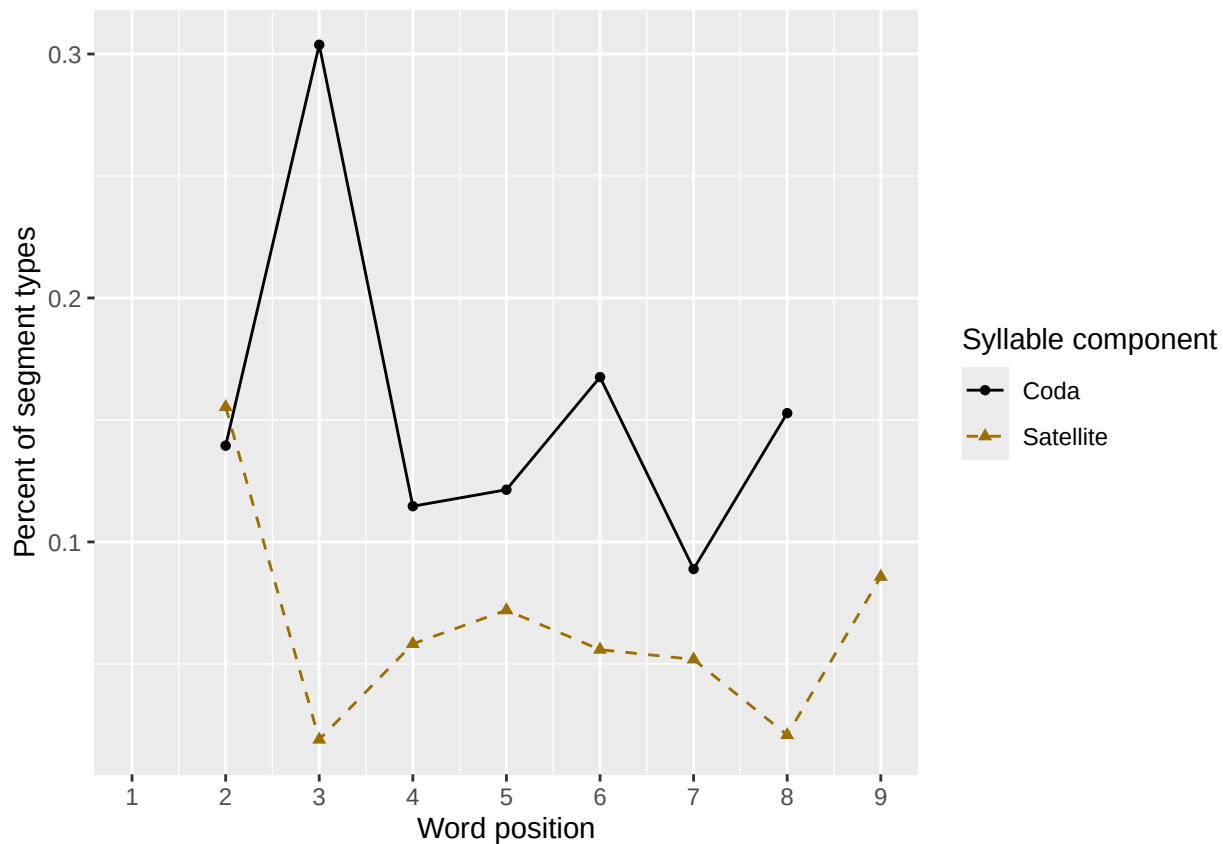
Warning: Removed 3 rows containing missing values or values outside the scale range (``geom_line()``).

Warning: Removed 3 rows containing missing values or values outside the scale range (``geom_point()``).

syll_comp_plot

```
## Warning: Removed 3 rows containing missing values or values outside the scale range (`geom_line()`).
```

Removed 3 rows containing missing values or values outside the scale range (``geom_point()``).



```
# len/pos table
pos_len_summary <- PosDat %>% group_by(stimlen,pos) %>% summarise(preserved = mean(preserved))

## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
min_preserved_raw <- min(as.numeric(unlist(pos_len_summary$preserved)))
max_preserved_raw <- max(as.numeric(unlist(pos_len_summary$preserved)))
preserved_range <- (max_preserved_raw - min_preserved_raw)
```

```

min_preserved <- min_preserved_raw - (0.1 * preserved_range)
max_preserved <- max_preserved_raw + (0.1 * preserved_range)
pos_len_table <- pos_len_summary %>% pivot_wider(names_from = pos, values_from = preserved)
write.csv(pos_len_table, paste0(TablesDir, CurPat, "_", RunLabel, "_", CurTask, "_preserved_percentages_by_len",
pos_len_table

```

```

## # A tibble: 7 x 10
## # Groups:   stimlen [7]
##   stimlen   `1`   `2`   `3`   `4`   `5`   `6`   `7`   `8`   `9`
##   <int> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl>
## 1      4 0.947 0.982 0.895 NA      NA      NA      NA      NA      NA
## 2      5 0.876 0.955 0.932 0.932 NA      NA      NA      NA      NA
## 3      6 0.920 0.880 0.844 0.872 0.874 NA      NA      NA      NA
## 4      7 0.862 0.935 0.813 0.867 0.906 0.833 NA      NA      NA
## 5      8 0.865 0.922 0.874 0.817 0.832 0.859 0.852 NA      NA
## 6      9 0.842 0.851 0.787 0.786 0.76  0.838 0.767 0.789 NA
## 7     10 0.873 0.886 0.725 0.652 0.710 0.712 0.783 0.853 0.8

```

```

# len/pos table
pos_len_N <- PosDat %>% group_by(stimlen, pos) %>% summarise(preserved = mean(preserved), N=n()) %>% dplyr::

```

`summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.

```

pos_len_N_table <- pos_len_N %>% pivot_wider(names_from = pos, values_from = N)
write.csv(pos_len_N_table, paste0(TablesDir, CurPat, "_", RunLabel, "_", CurTask,
"_N_by_len_position.csv"))
pos_len_N_table

```

```

## # A tibble: 7 x 10
## # Groups:   stimlen [7]
##   stimlen   `1`   `2`   `3`   `4`   `5`   `6`   `7`   `8`   `9`
##   <int> <int> <int> <int> <int> <int> <int> <int> <int> <int>
## 1      4    57    56    57    NA    NA    NA    NA    NA    NA
## 2      5    89    88    88    88    NA    NA    NA    NA    NA
## 3      6   112   108   109   109   111    NA    NA    NA    NA
## 4      7   109   107   107   105   106   108    NA    NA    NA
## 5      8   133   128   127   126   125   128   128    NA    NA
## 6      9    76    74    75    70    75    74    73    76    NA
## 7     10    71    70    69    69    69    66    69    68    70

```

```

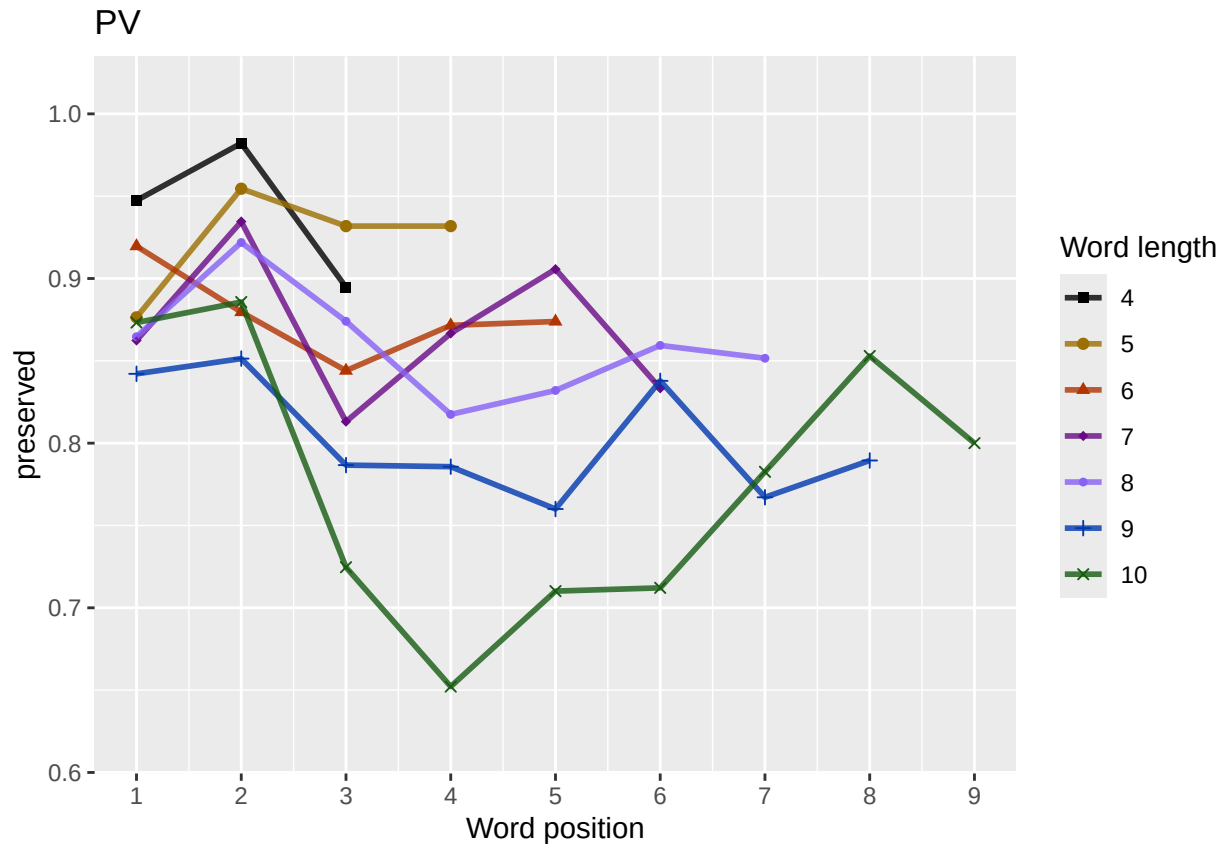
obs_linetypes <- c("solid", "solid", "solid", "solid",
"solid", "solid", "solid", "solid", "solid")
pred_linetypes <- "dashed"
pos_len_summary$stimlen <- factor(pos_len_summary$stimlen)
pos_len_summary$pos <- factor(pos_len_summary$pos)
# len_pos_plot <- ggplot(pos_len_summary, aes(x=pos, y=preserved, group=stimlen, shape=stimlen, color=stimlen))
len_pos_plot <- plot_len_pos_obs_predicted(PosDat,
paste0(CurPat),
NULL,
c(min_preserved, max_preserved),
palette_values,
shape_values,
obs_linetypes,
pred_linetypes)

```

`summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.

```
## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
## Scale for y is already present. Adding another scale for y, which will replace the existing scale.
```

```
ggsave(paste0(FigDir, CurPat, "_", RunLabel, "_", CurTask,
              "_percent_preserved_by_length_pos.png"),
       plot=len_pos_plot, device="png", unit="cm", width=15, height=11)
len_pos_plot
```



Length and position

```
# length and position
```

```
LPMModelEquations<-c("preserved ~ 1",
                      "preserved ~ stimlen",
                      "preserved ~ pos",
                      "preserved ~ stimlen + pos",
                      "preserved ~ stimlen*pos",
                      "preserved ~ I(pos^2)+pos",
                      "preserved ~ stimlen + I(pos^2) + pos",
                      "preserved ~ stimlen * (I(pos^2) + pos)"
)
```

```
LPres<-TestModels(LPMModelEquations,PosDat)
```

```
## *****
```

```
## model index: 8
```

```
##
```

```
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
```

```
## data = PosDat)
```

```

##
## Coefficients:
##      (Intercept)      stimlen      I(pos^2)      pos  stimlen:I(pos^2)      stiml
##      2.44802      -0.01839      -0.05409      0.54613      0.01020      -0
##
## Degrees of Freedom: 3822 Total (i.e. Null); 3817 Residual
## Null Deviance:      3213
## Residual Deviance: 3131 AIC: 3143
## log likelihood: -1565.584
## Nagelkerke R2: 0.03728263
## % pres/err predicted correctly: -944.8066
## % of predictable range [ (model-null)/(1-null) ]: 0.02213854
## *****
## model index: 7
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##      data = PosDat)
##
## Coefficients:
##      (Intercept)      stimlen      I(pos^2)      pos
##      4.0295      -0.2204      0.0278      -0.2849
##
## Degrees of Freedom: 3822 Total (i.e. Null); 3819 Residual
## Null Deviance:      3213
## Residual Deviance: 3135 AIC: 3143
## log likelihood: -1567.7
## Nagelkerke R2: 0.03537525
## % pres/err predicted correctly: -946.4714
## % of predictable range [ (model-null)/(1-null) ]: 0.02041734
## *****
## model index: 4
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##      data = PosDat)
##
## Coefficients:
##      (Intercept)      stimlen      pos
##      3.50134      -0.20413      -0.04173
##
## Degrees of Freedom: 3822 Total (i.e. Null); 3820 Residual
## Null Deviance:      3213
## Residual Deviance: 3144 AIC: 3150
## log likelihood: -1571.958
## Nagelkerke R2: 0.03153109
## % pres/err predicted correctly: -948.9661
## % of predictable range [ (model-null)/(1-null) ]: 0.01783808
## *****
## model index: 2
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##      data = PosDat)
##
## Coefficients:
##      (Intercept)      stimlen

```

```

##      3.5022      -0.2253
##
## Degrees of Freedom: 3822 Total (i.e. Null); 3821 Residual
## Null Deviance:      3213
## Residual Deviance: 3147 AIC: 3151
## log likelihood: -1573.724
## Nagelkerke R2: 0.02993472
## % pres/err predicted correctly: -949.9026
## % of predictable range [ (model-null)/(1-null) ]: 0.01686983
## *****
## model index: 5
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept)      stimlen      pos      stimlen:pos
##    3.690349    -0.226539    -0.099956     0.006713
##
## Degrees of Freedom: 3822 Total (i.e. Null); 3819 Residual
## Null Deviance:      3213
## Residual Deviance: 3144 AIC: 3152
## log likelihood: -1571.857
## Nagelkerke R2: 0.03162223
## % pres/err predicted correctly: -948.9962
## % of predictable range [ (model-null)/(1-null) ]: 0.01780695
## *****
## model index: 6
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept)      I(pos^2)      pos
##    2.38854     0.01558    -0.24228
##
## Degrees of Freedom: 3822 Total (i.e. Null); 3820 Residual
## Null Deviance:      3213
## Residual Deviance: 3186 AIC: 3192
## log likelihood: -1592.976
## Nagelkerke R2: 0.01243148
## % pres/err predicted correctly: -959.4523
## % of predictable range [ (model-null)/(1-null) ]: 0.006996463
## *****
## model index: 3
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept)      pos
##    2.1546     -0.1037
##
## Degrees of Freedom: 3822 Total (i.e. Null); 3821 Residual

```



```
## Null Deviance:      3213
## Residual Deviance: 3189 AIC: 3193
## log likelihood: -1594.357
## Nagelkerke R2: 0.01116939
## % pres/err predicted correctly: -960.0668
## % of predictable range [ (model-null)/(1-null) ]: 0.006361135
## *****
## model index: 1
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept)
## 1.746
##
## Degrees of Freedom: 3822 Total (i.e. Null); 3822 Residual
## Null Deviance:      3213
## Residual Deviance: 3213 AIC: 3215
## log likelihood: -1606.533
## Nagelkerke R2: 1.952945e-16
## % pres/err predicted correctly: -966.2195
## % of predictable range [ (model-null)/(1-null) ]: 0
## *****
```

```
BestLPModel<-LPRes$ModelResult[[1]]
BestLPModelFormula<-LPRes$Model[[1]]

LPAICSummary<-data.frame(Model=LPRes$Model,
                          AIC=LPRes$AIC,
                          row.names = LPRes$Model)
LPAICSummary$DeltaAIC<-LPAICSummary$AIC-LPAICSummary$AIC[1]
LPAICSummary$AICexp<-exp(-0.5*LPAICSummary$DeltaAIC)
LPAICSummary$AICwt<-LPAICSummary$AICexp/sum(LPAICSummary$AICexp)
LPAICSummary$NagR2<-LPRes$NagR2

LPAICSummary <- merge(LPAICSummary,LPRes$CoefficientValues, by='row.names',sort=FALSE)
LPAICSummary <- subset(LPAICSummary, select = -c(Row.names))

write.csv(LPAICSummary,paste0(TablesDir,CurPat,"_",RunLabel,"_",CurTask,
                              "_len_pos_model_summary.csv"),row.names = FALSE)

kable(LPAICSummary)
```

Model	AIC	DeltaAIC	AICexp	AICwt	NagR2 (Intercept)	stimlen	pos	stimlen:I(pos)	stimlen:I(pos^2)	stimlen:I(pos^2)
preserved ~ stimlen * (I(pos^2) + pos)	3143.167	0.000000	0.000000	0.051171	0.803728	26448020	- 0.5461292	-	-	0.0101957
							0.0183881	0.1044930	0.0540943	
preserved ~ stimlen + I(pos^2) + pos	3143.400	0.232786	0.189012	0.534554	0.903537	12029486	-	-	NA	0.0278038
							0.2203975	0.2848663		NA
preserved ~ stimlen + pos	3149.916	6.749344	0.034220	0.030175	0.580315	31501336	-	-	NA	NA
							0.2041289	0.0417311		
preserved ~ stimlen	3151.448	8.280482	0.015919	0.000814	0.600299	337502214	-	NA	NA	NA
							0.2253417			

Model	AIC	DeltaAIC	AICexp	AICwt	NagR2	(Intercept)	stimlen	pos	stimlen:I(pos^2)	I(pos^2)	stimlen:I(pos^2)
preserved ~ stimlen * pos	3151.715	5.547664	430139283	00712740316223	690349	-	-	0.0067128	NA	NA	NA
							0.2265390	0.0999564			
preserved ~ I(pos^2) + pos	3191.952	8.784693	0000000000000000	0001243253	88538	NA	-	NA	0.0155804	NA	NA
							0.2422753				
preserved ~ pos	3192.714	9.546142	0000000000000000	0001116241	54613	NA	-	NA	NA	NA	NA
							0.1037395				
preserved ~ 1	3215.067	1.898243	8000000000000000	0000000000	00745826	NA	NA	NA	NA	NA	NA

```
print(BestLPModelFormula)
```

```
## [1] "preserved ~ stimlen * (I(pos^2) + pos)"
```

```
print(BestLPModel)
```

```
##
```

```
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
```

```
## data = PosDat)
```

```
##
```

```
## Coefficients:
```

```
## (Intercept) stimlen I(pos^2) pos stimlen:I(pos^2) stimlen:I(pos^2)
```

```
## 2.44802 -0.01839 -0.05409 0.54613 0.01020 -0.01020
```

```
##
```

```
## Degrees of Freedom: 3822 Total (i.e. Null); 3817 Residual
```

```
## Null Deviance: 3213
```

```
## Residual Deviance: 3131 AIC: 3143
```

```
PosDat$LPFitted<-fitted(BestLPModel)
```

```
fitted_len_pos_plot <- plot_len_pos_obs_predicted(PosDat, PosDat$patient[1],  
NULL,palette_values=palette_values)
```

```
## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
```

```
## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
```

```
# len/pos table
```

```
fitted_pos_len_summary <- PosDat %>% group_by(stimlen,pos) %>% summarise(fitted = mean(LPfitted))
```

```
## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
```

```
fitted_pos_len_table <- fitted_pos_len_summary %>% pivot_wider(names_from = pos, values_from = fitted)  
fitted_pos_len_table
```

```
## # A tibble: 7 x 10
```

```
## # Groups: stimlen [7]
```

```
## stimlen `1` `2` `3` `4` `5` `6` `7` `8` `9`
```

```
## <int> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl>
```

```
## 1 4 0.923 0.929 0.933 NA NA NA NA NA NA
```

```
## 2 5 0.915 0.916 0.917 0.917 NA NA NA NA NA
```

```
## 3 6 0.906 0.901 0.896 0.894 0.892 NA NA NA NA
```

```
## 4 7 0.896 0.883 0.872 0.865 0.861 0.862 NA NA NA
```

```
## 5 8 0.885 0.862 0.843 0.829 0.823 0.825 0.835 NA NA
```

```
## 6 9 0.873 0.838 0.808 0.787 0.778 0.781 0.797 0.823 NA
```

```
## 7 10 0.860 0.811 0.768 0.738 0.724 0.730 0.754 0.792 0.839
```

```
fitted_pos_len_summary$stimlen<-factor(fitted_pos_len_summary$stimlen)
```

```
fitted_pos_len_summary$pos<-factor(fitted_pos_len_summary$pos)
```

```

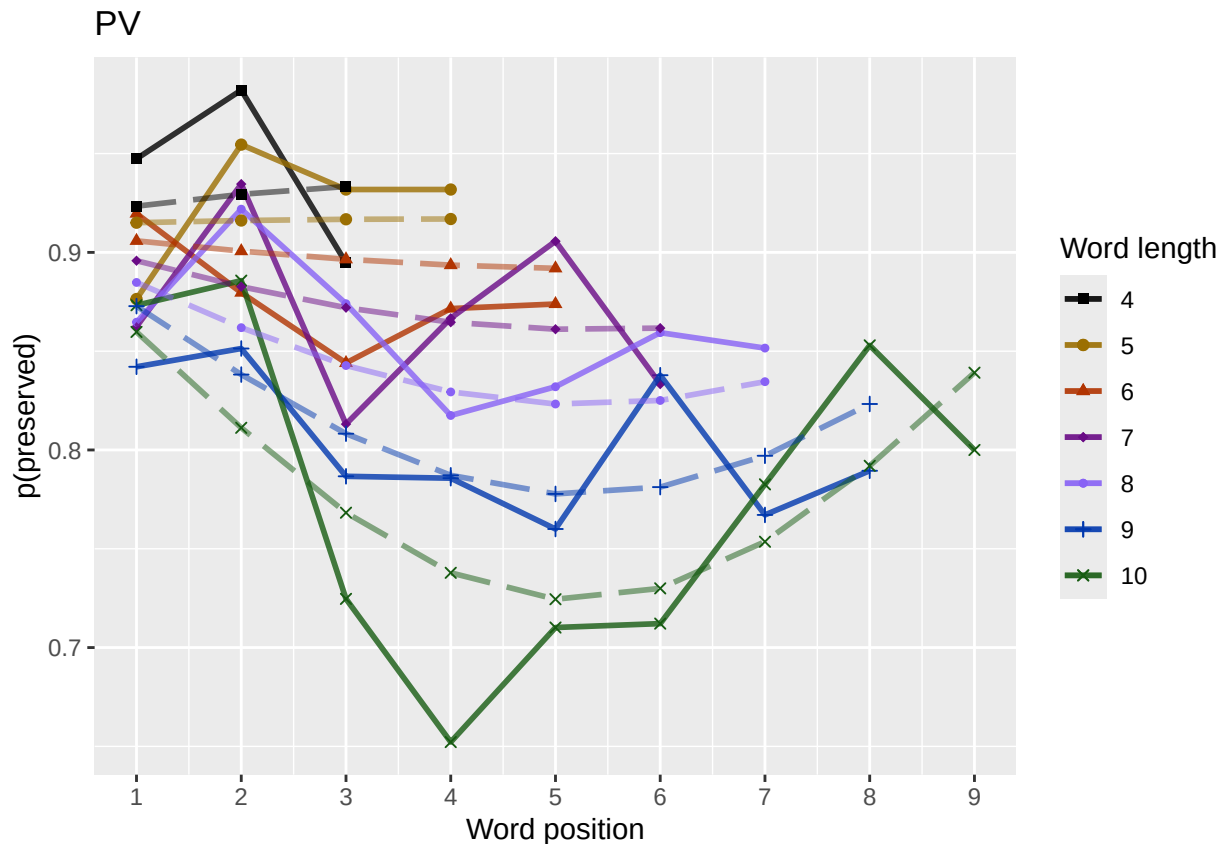
# fitted_len_pos_plot <- ggplot(pos_len_summary, aes(x=pos, y=preserved, group=stimlen, shape=stimlen, color=stimlen))
# fitted_len_pos_plot <- fitted_len_pos_plot + geom_line(data=fitted_pos_len_summary, aes(x=pos, y=fitted, group=stimlen, shape=stimlen, color=stimlen))
# geom_point(data=fitted_pos_len_summary, aes(x=pos, y=fitted, group=stimlen, shape=stimlen)) + ggtitle(paste0("Patient", patient_id))

fitted_len_pos_plot <- plot_len_pos_obs_predicted(PosDat,
  paste0(PosDat$patient[1]),
  "LPFitted",
  NULL,
  palette_values,
  shape_values,
  obs_linetypes,
  pred_linetypes = c("longdash")
)

## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.

ggsave(paste0(FigDir, CurPat, "_", RunLabel, "_", CurTask, "_percent_preserved_by_length_pos_wfit.png"), plot=fitted_len_pos_plot

```



length and position without fragments to see if this changes position² influence

```

# first number responses, then count resp with fragments - below we will eliminate fragments
# and re-run models

# number responses
resp_num <- 0

```

```

prev_pos<-9999 # big number to initialize (so first position is smaller)
resp_num_array <- integer(length = nrow(PosDat))
for(i in seq(1,nrow(PosDat))){
  if(PosDat$pos[i] <= prev_pos){ # equal for resp length 1
    resp_num <- resp_num + 1
  }
  resp_num_array[i]<-resp_num
  prev_pos <- PosDat$pos[i]
}
PosDat$resp_num <- resp_num_array

# count responses with fragments
resp_with_frag <- PosDat %>% group_by(resp_num) %>% summarise(frag = as.logical(sum(fragment)))
num_frag <- resp_with_frag %>% summarise(frag_sum = sum(frag), N=n())
num_frag

## # A tibble: 1 x 2
##   frag_sum      N
##   <int> <int>
## 1      34   651

num_frag$percent_with_frag[1] <- (num_frag$frag_sum[1] / num_frag$N[1])*100

## Warning: Unknown or uninitialised column: `percent_with_frag`.

print(sprintf("The number of responses with fragments was %d / %d = %.2f percent",
  num_frag$frag_sum[1],num_frag$N[1],num_frag$percent_with_frag[1]))

## [1] "The number of responses with fragments was 34 / 651 = 5.22 percent"

write.csv(num_frag,paste0(TablesDir,CurPat,"_",RunLabel,"_",
  CurTask,"_percent_fragments.csv"),row.names = FALSE)

# length and position models for data without fragments
NoFragData <- PosDat %>% filter(fragment != 1)

NoFrag_LPRes<-TestModels(LPModelEquations,NoFragData)

## *****
## model index: 8
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##   data = PosDat)
##
## Coefficients:
##   (Intercept)      stimlen      I(pos^2)      pos  stimlen:I(pos^2)      stimlen
##   2.28036      0.01706      -0.03450      0.60833      0.01082      -0.01082
##
## Degrees of Freedom: 3749 Total (i.e. Null); 3744 Residual
## Null Deviance: 2926
## Residual Deviance: 2850 AIC: 2862
## log likelihood: -1425.065
## Nagelkerke R2: 0.03710712
## % pres/err predicted correctly: -839.2798
## % of predictable range [ (model-null)/(1-null) ]: 0.0221573
## *****

```

```

## model index: 7
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept)      stimlen      I(pos^2)          pos
##      4.25382      -0.24221      0.04584      -0.35387
##
## Degrees of Freedom: 3749 Total (i.e. Null); 3746 Residual
## Null Deviance:      2926
## Residual Deviance: 2858 AIC: 2866
## log likelihood: -1428.823
## Nagelkerke R2: 0.0334778
## % pres/err predicted correctly: -841.7768
## % of predictable range [ (model-null)/(1-null) ]: 0.01925147
## *****
## model index: 2
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept)      stimlen
##      3.4723      -0.2046
##
## Degrees of Freedom: 3749 Total (i.e. Null); 3748 Residual
## Null Deviance:      2926
## Residual Deviance: 2878 AIC: 2882
## log likelihood: -1438.865
## Nagelkerke R2: 0.02374536
## % pres/err predicted correctly: -847.3578
## % of predictable range [ (model-null)/(1-null) ]: 0.0127568
## *****
## model index: 4
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept)      stimlen          pos
##      3.46783      -0.21971      0.03227
##
## Degrees of Freedom: 3749 Total (i.e. Null); 3747 Residual
## Null Deviance:      2926
## Residual Deviance: 2876 AIC: 2882
## log likelihood: -1437.942
## Nagelkerke R2: 0.02464142
## % pres/err predicted correctly: -846.8877
## % of predictable range [ (model-null)/(1-null) ]: 0.01330384
## *****
## model index: 5
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",

```

```

##      data = PosDat)
##
## Coefficients:
## (Intercept)      stimlen          pos  stimlen:pos
##      3.613855    -0.237063    -0.015355     0.005509
##
## Degrees of Freedom: 3749 Total (i.e. Null);  3746 Residual
## Null Deviance:      2926
## Residual Deviance: 2876  AIC: 2884
## log likelihood:  -1437.884
## Nagelkerke R2:  0.02469776
## % pres/err predicted correctly:  -846.8827
## % of predictable range [ (model-null)/(1-null) ]:  0.01330973
## *****
## model index:  6
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##      data = PosDat)
##
## Coefficients:
## (Intercept)      I(pos^2)          pos
##      2.44042      0.03217     -0.30485
##
## Degrees of Freedom: 3749 Total (i.e. Null);  3747 Residual
## Null Deviance:      2926
## Residual Deviance: 2915  AIC: 2921
## log likelihood:  -1457.626
## Nagelkerke R2:  0.005421998
## % pres/err predicted correctly:  -855.7872
## % of predictable range [ (model-null)/(1-null) ]:  0.002947435
## *****
## model index:  1
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##      data = PosDat)
##
## Coefficients:
## (Intercept)
##      1.883
##
## Degrees of Freedom: 3749 Total (i.e. Null);  3749 Residual
## Null Deviance:      2926
## Residual Deviance: 2926  AIC: 2928
## log likelihood:  -1463.141
## Nagelkerke R2:  0
## % pres/err predicted correctly:  -858.32
## % of predictable range [ (model-null)/(1-null) ]:  0
## *****
## model index:  3
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##      data = PosDat)
##
## Coefficients:

```

```
## (Intercept)          pos
##      1.99943      -0.03077
##
## Degrees of Freedom: 3749 Total (i.e. Null);  3748 Residual
## Null Deviance:      2926
## Residual Deviance: 2924  AIC: 2928
## log likelihood:  -1462.231
## Nagelkerke R2:   0.0008955866
## % pres/err predicted correctly:  -857.9233
## % of predictable range [ (model-null)/(1-null) ]:  0.0004616789
## *****
```

```
NoFragBestLPModel<-NoFrag_LPRes$ModelResult[[1]]
NoFragBestLPModelFormula<-NoFrag_LPRes$Model[[1]]

NoFragLPAICSummary<-data.frame(Model=NoFrag_LPRes$Model,
                                AIC=NoFrag_LPRes$AIC,
                                row.names = NoFrag_LPRes$Model)
NoFragLPAICSummary$DeltaAIC<-NoFragLPAICSummary$AIC-NoFragLPAICSummary$AIC[1]
NoFragLPAICSummary$AICexp<-exp(-0.5*NoFragLPAICSummary$DeltaAIC)
NoFragLPAICSummary$AICwt<-NoFragLPAICSummary$AICexp/sum(NoFragLPAICSummary$AICexp)
NoFragLPAICSummary$NagR2<-NoFrag_LPRes$NagR2

NoFragLPAICSummary <- merge(NoFragLPAICSummary,NoFrag_LPRes$CoefficientValues, by='row.names',sort=FALSE)
NoFragLPAICSummary <- subset(NoFragLPAICSummary, select = -c(Row.names))

write.csv(NoFragLPAICSummary,paste0(TablesDir,
                                    CurPat,"_",RunLabel,"_",
                                    CurTask,
                                    "_no_fragments_len_pos_model_summary.csv"),
          row.names = FALSE)
kable(NoFragLPAICSummary)
```

Model	AIC	DeltaAIC	AICexp	AICwt	NagR2	(Intercept)	stimlen	pos	stimlen:I(pos)	stimlen:I(pos^2)	stimlen:I(pos^2)
preserved ~ stimlen * (I(pos^2) + pos)	2862.129	0.000001	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000
preserved ~ stimlen + I(pos^2) + pos	2865.645	3.516920	0.172310	0.014696	0.003347	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000
preserved ~ stimlen	2881.720	19.60037	0.000005	0.000004	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000
preserved ~ stimlen + pos	2881.885	19.75579	0.000005	0.000004	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000
preserved ~ stimlen * pos	2883.762	21.63977	0.000002	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000
preserved ~ I(pos^2) + pos	2921.251	59.12223	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000
preserved ~ 1	2928.283	66.15358	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000
preserved ~ pos	2928.463	66.33370	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000

```
# plot no fragment data
```

```
NoFragData$LPFitted<-fitted(NoFragBestLPModel)
nofrag_obs_len_pos_plot <- plot_len_pos_obs_predicted(NoFragData, NoFragData$patient[1],
  NULL,palette_values=palette_values)
```

```
## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
```

```
# len/pos table
```

```
nofrag_fitted_pos_len_summary <- NoFragData %>% group_by(stimlen,pos) %>% summarise(fitted = mean(LPfit
```

```
## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
```

```
nofrag_fitted_pos_len_table <- nofrag_fitted_pos_len_summary %>% pivot_wider(names_from = pos, values_f
nofrag_fitted_pos_len_table
```

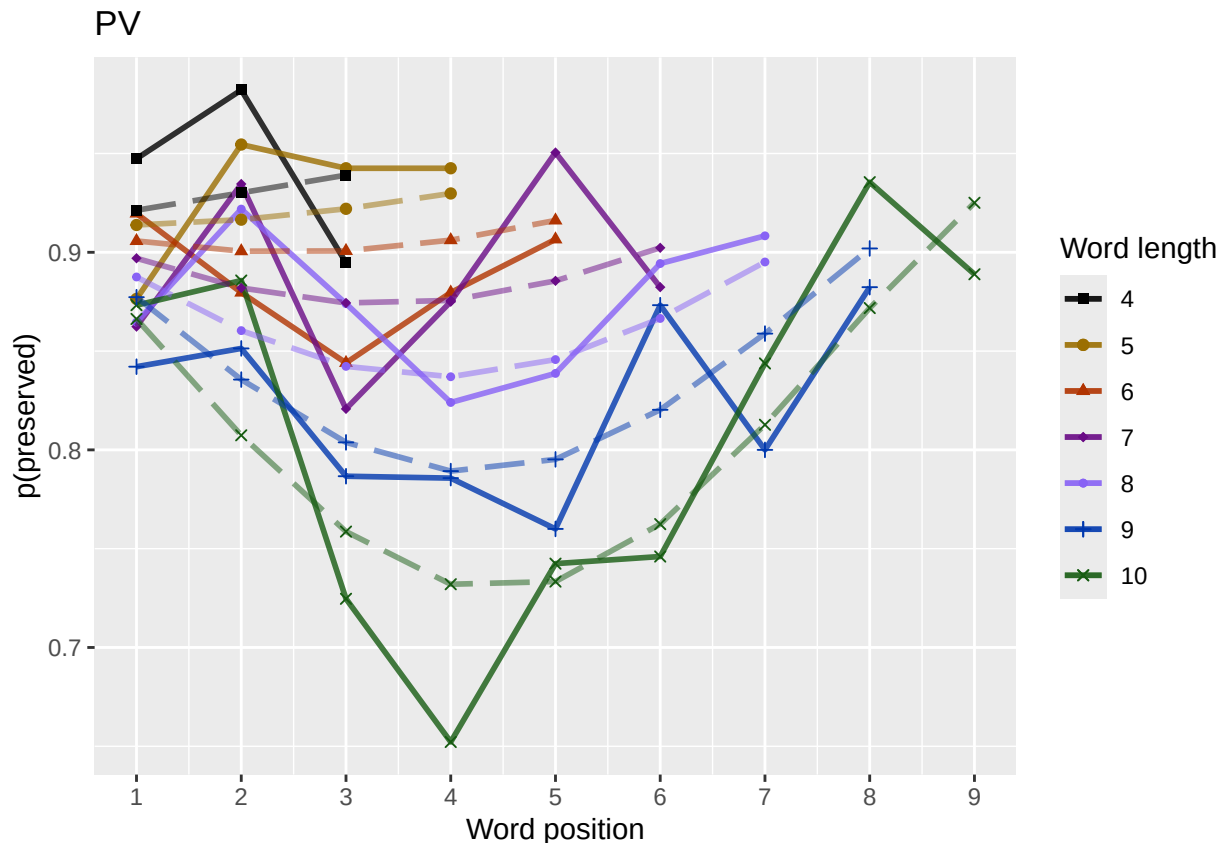
```
## # A tibble: 7 x 10
## # Groups:   stimlen [7]
##   stimlen   `1`   `2`   `3`   `4`   `5`   `6`   `7`   `8`   `9`
##   <int> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl>
## 1      4 0.921 0.930 0.939 NA      NA      NA      NA      NA      NA
## 2      5 0.914 0.917 0.922 0.930 NA      NA      NA      NA      NA
## 3      6 0.906 0.901 0.901 0.906 0.916 NA      NA      NA      NA
## 4      7 0.897 0.882 0.874 0.876 0.886 0.902 NA      NA      NA
## 5      8 0.888 0.860 0.842 0.837 0.846 0.867 0.895 NA      NA
## 6      9 0.877 0.836 0.804 0.789 0.795 0.820 0.859 0.902 NA
## 7     10 0.866 0.807 0.759 0.732 0.733 0.762 0.813 0.872 0.925
```

```
fitted_pos_len_summary$stimlen<-factor(nofrag_fitted_pos_len_summary$stimlen)
fitted_pos_len_summary$pos<-factor(nofrag_fitted_pos_len_summary$pos)
# fitted_len_pos_plot <- ggplot(pos_len_summary,aes(x=pos,y=preserved,group=stimlen,shape=stimlen,color=stimlen)) +
#   geom_line(data=fitted_pos_len_summary, aes(x=pos,y=fitted,group=stimlen,shape=stimlen)) + ggtitle(paste0("Patient",patient[1]))
#   geom_point(data=fitted_pos_len_summary, aes(x=pos,y=fitted,group=stimlen,shape=stimlen)) + ggtitle(paste0("Patient",patient[1]))
```

```
nofrag_fitted_len_pos_plot <- plot_len_pos_obs_predicted(NoFragData,
  paste0(NoFragData$patient[1]),
  "LPFitted",
  NULL,
  palette_values,
  shape_values,
  obs_linetypes,
  pred_linetypes = c("longdash")
)
```

```
## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
```

```
ggsave(paste0(FigDir,CurPat,"_",RunLabel,"_",CurTask,
  "no_fragments_percent_preserved_by_length_pos_wfit.png"),
  plot=nofrag_fitted_len_pos_plot,
  device="png",unit="cm",
  width=15,height=11)
nofrag_fitted_len_pos_plot
```

back to full data

```
results_report_DF <- AddReportLine(results_report_DF,"min preserved",min_preserved)
results_report_DF <- AddReportLine(results_report_DF,"max preserved",max_preserved)
print(sprintf("Min/max preserved range: %.2f - %.2f",min_preserved,max_preserved))
```

```
## [1] "Min/max preserved range: 0.62 - 1.02"
```

```
# find the average difference between lengths and the average difference
# between positions taking the other factor into account
```

```
# choose fitted or raw -- we use fitted values because they are smoothed averages
# that take into account the influence of the other variable
```

```
table_to_use <- fitted_pos_len_table
# take away column with length information
table_to_use <- table_to_use[,2:ncol(table_to_use)]
```

```
# take averages for positions
```

```
# don't want downward estimates influenced by return upward of U
```

```
# therefore, for downward influence, use only the values before the min
```

```
# take the difference between each value (differences between position proportion correct) **NOTE** pro
```

```
# average the difference in probabilities
```

```
# in case min is close to the end or we are not using a min (for non-U-shaped)
```

```
# functions, we need to get differences _first_ and then average those (e.g.
```

```
# if there is an effect of length and we just average position data,
```

```
# later positions) will have a lower average (because of the length effect)
```

```
# even if all position functions go upward
```

```

table_pos_diffs <- t(diff(t(as.matrix(table_to_use))))
ncol_pos_diff_matrix <- ncol(table_pos_diffs)
first_col_mean <- mean(as.numeric(unlist(table_pos_diffs[,1])))
potential_u_shape <- 0
if(first_col_mean < 0){ # downward, so potential U-shape
  # take only negative values from the table
  filtered_table_pos_diffs<-table_pos_diffs
  filtered_table_pos_diffs[table_pos_diffs >= 0] <- NA
  potential_u_shape <- 1
}else{
  # upward - use the positive values
  # take only negative values from the table
  filtered_table_pos_diffs<-table_pos_diffs
  filtered_table_pos_diffs[table_pos_diffs <= 0] <- NA
}
average_pos_diffs <- apply(filtered_table_pos_diffs,2,mean,na.rm=TRUE)
OA_mean_pos_diff <- mean(average_pos_diffs,na.rm=TRUE)

table_len_diffs <- diff(as.matrix(table_to_use))
average_len_diffs <- apply(table_len_diffs,1,mean,na.rm=TRUE)
OA_mean_len_diff <- mean(average_len_diffs,na.rm=TRUE)
CurrentLabel<-"mean change in probability for each additional length: "
print(CurrentLabel)

## [1] "mean change in probability for each additional length: "
print(OA_mean_len_diff)

## [1] -0.02558013
results_report_DF <- AddReportLine(results_report_DF, CurrentLabel,OA_mean_len_diff)

CurrentLabel<-"mean change in probability for each additional position (excluding U-shape): "
print(CurrentLabel)

## [1] "mean change in probability for each additional position (excluding U-shape): "
print(OA_mean_pos_diff)

## [1] -0.01702211
results_report_DF <- AddReportLine(results_report_DF, CurrentLabel,OA_mean_pos_diff)

if(potential_u_shape){ # downward, so potential U-shape
  # take only positive values from the table
  filtered_pos_upward_u<-table_pos_diffs
  filtered_pos_upward_u[table_pos_diffs <= 0] <- NA
  average_pos_u_diffs <- apply(filtered_pos_upward_u,
    2,mean,na.rm=TRUE)
  OA_mean_pos_u_diff <- mean(average_pos_u_diffs,na.rm=TRUE)
  if(is.nan(OA_mean_pos_u_diff) | (OA_mean_pos_u_diff <= 0)){
    print("No U-shape in this participant")
    results_report_DF <- AddReportLine(results_report_DF, "U-shape", 0)
    potential_u_shape <- FALSE
  }else{
    results_report_DF <- AddReportLine(results_report_DF, "U-shape", 1)
  }
}

```

```

    CurrentLabel<-"Average upward change after U minimum"
    print(CurrentLabel)
    print(OA_mean_pos_u_diff)
    results_report_DF <- AddReportLine(results_report_DF, CurrentLabel, OA_mean_pos_u_diff)

    CurrentLabel<-"Proportion of average downward change"
    prop_ave_down <- abs(OA_mean_pos_u_diff/OA_mean_pos_diff)
    results_report_DF <- AddReportLine(results_report_DF, CurrentLabel, prop_ave_down)
  }
}else{
  print("No U-shape in this participant")
  results_report_DF <- AddReportLine(results_report_DF, "U-shape", 0)
}

## [1] "Average upward change after U minimum"
## [1] 0.0149415

if(potential_u_shape){
  n_rows<-nrow(table_to_use)
  downward_dist <- numeric(n_rows)
  upward_dist <- numeric(n_rows)
  for(i in seq(1,n_rows)){
    current_row <- as.numeric(unlist(table_to_use[i,1:9]))
    current_row <- current_row[!is.na(current_row)]
    current_row_len <- length(current_row)
    row_min <- min(current_row,na.rm=TRUE)
    min_pos <- which(current_row == row_min)
    left_max <- max(current_row[1:min_pos],na.rm=TRUE)
    right_max <- max(current_row[min_pos:current_row_len])
    left_diff <- left_max - row_min
    right_diff <- right_max - row_min
    downward_dist[i]<-left_diff
    upward_dist[i]<-right_diff
  }
  print("differences from left max to min for each row: ")
  print(downward_dist)
  print("differences from min to right max for each row: ")
  print(upward_dist)

  # Use the max return upward (min of upward_dist) and then the
  # downward distance from the left for the same row

  print(" ")
  biggest_return_upward_row <- which(upward_dist == max(upward_dist))
  CurrentLabel <- "row with biggest return upward"
  print(CurrentLabel)
  print(biggest_return_upward_row)
  results_report_DF <- AddReportLine(results_report_DF, CurrentLabel, biggest_return_upward_row)

  print(" ")
  CurrentLabel<-"downward distance for row with the largest upward value"
  print(CurrentLabel)
  print(downward_dist[biggest_return_upward_row])
  results_report_DF <- AddReportLine(results_report_DF, CurrentLabel, downward_dist[biggest_return_upward_row])
}

```

```

CurrentLabel<-"return upward value"
print(upward_dist[biggest_return_upward_row])
results_report_DF <- AddReportLine(results_report_DF,
                                   CurrentLabel,
                                   upward_dist[biggest_return_upward_row])

print(" ")
# percentage return upward
percentage_return_upward <- upward_dist[biggest_return_upward_row]/downward_dist[biggest_return_upward_row]
CurrentLabel <- "Return upward as a proportion of the downward distance:"
print(CurrentLabel)
print(percentage_return_upward)
results_report_DF <- AddReportLine(results_report_DF, CurrentLabel,
                                   percentage_return_upward)
}else{
  print("no U-shape in this participant")
}

## [1] "differences from left max to min for each row: "
## [1] 0.00000000 0.00000000 0.01392315 0.03467174 0.06152435 0.09503442 0.13528850
## [1] "differences from min to right max for each row: "
## [1] 0.009948873 0.001866779 0.000000000 0.000561953 0.011295743 0.045521108 0.114671637
## [1] " "
## [1] "row with biggest return upward"
## [1] 7
## [1] " "
## [1] "downward distance for row with the largest upward value"
## [1] 0.1352885
## [1] 0.1146716
## [1] " "
## [1] "Return upward as a proportion of the downward distance:"
## [1] 0.8476081

FLPModelEquations<-c(
  # models with log frequency, but no three-way interactions (due to difficulty interpreting and small
  "preserved ~ stimlen*log_freq",
  "preserved ~ stimlen+log_freq",
  "preserved ~ pos*log_freq",
  "preserved ~ pos+log_freq",
  "preserved ~ stimlen*log_freq + pos*log_freq",
  "preserved ~ stimlen*log_freq + pos",
  "preserved ~ stimlen + pos*log_freq",
  "preserved ~ stimlen + pos + log_freq",
  "preserved ~ (I(pos^2)+pos)*log_freq",
  "preserved ~ stimlen*log_freq + (I(pos^2) + pos)*log_freq",
  "preserved ~ stimlen*log_freq + I(pos^2) + pos",
  "preserved ~ stimlen + (I(pos^2) + pos)*log_freq",
  "preserved ~ stimlen + I(pos^2) + pos + log_freq",

  # models without frequency
  "preserved ~ 1",
  "preserved ~ stimlen",
  "preserved ~ pos",
  "preserved ~ stimlen + pos",

```

```

    "preserved ~ stimlen*pos",
    "preserved ~ I(pos^2)+pos",
    "preserved ~ stimlen + I(pos^2) + pos",
    "preserved ~ stimlen * (I(pos^2) + pos)"
)

FLPRes<-TestModels(FLPModelEquations,PosDat)

## *****
## model index:  11
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##          data = PosDat)
##
## Coefficients:
##      (Intercept)      stimlen      log_freq      I(pos^2)      pos  stimlen:log
##           3.75865      -0.18782           0.41749           0.02672      -0.27567           -0
##
## Degrees of Freedom: 3822 Total (i.e. Null);  3817 Residual
## Null Deviance:      3213
## Residual Deviance: 3110  AIC: 3122
## log likelihood:  -1554.932
## Nagelkerke R2:  0.0468506
## % pres/err predicted correctly:  -940.9082
## % of predictable range [ (model-null)/(1-null) ]:  0.0261691
## *****
## model index:  10
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##          data = PosDat)
##
## Coefficients:
##      (Intercept)      stimlen      log_freq      I(pos^2)      pos  stim
##           3.745878      -0.191005           0.338013           0.025703      -0.260844
##      log_freq:pos
##           0.048667
##
## Degrees of Freedom: 3822 Total (i.e. Null);  3815 Residual
## Null Deviance:      3213
## Residual Deviance: 3108  AIC: 3124
## log likelihood:  -1553.989
## Nagelkerke R2:  0.04769474
## % pres/err predicted correctly:  -940.3185
## % of predictable range [ (model-null)/(1-null) ]:  0.02677882
## *****
## model index:  13
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##          data = PosDat)
##
## Coefficients:
## (Intercept)      stimlen      I(pos^2)      pos      log_freq
##           3.7854      -0.1863           0.0277      -0.2846           0.1103
##

```

```

## Degrees of Freedom: 3822 Total (i.e. Null); 3818 Residual
## Null Deviance: 3213
## Residual Deviance: 3116 AIC: 3126
## log likelihood: -1557.802
## Nagelkerke R2: 0.04427782
## % pres/err predicted correctly: -941.6948
## % of predictable range [ (model-null)/(1-null) ]: 0.02535583
## *****
## model index: 6
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept) stimlen log_freq pos stimlen:log_freq
## 3.24922 -0.17195 0.42960 -0.04224 -0.03944
##
## Degrees of Freedom: 3822 Total (i.e. Null); 3818 Residual
## Null Deviance: 3213
## Residual Deviance: 3118 AIC: 3128
## log likelihood: -1558.821
## Nagelkerke R2: 0.04336294
## % pres/err predicted correctly: -943.2432
## % of predictable range [ (model-null)/(1-null) ]: 0.02375495
## *****
## model index: 12
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept) stimlen I(pos^2) pos log_freq I(pos
## 3.750828 -0.186443 0.025137 -0.262697 0.010015
##
## Degrees of Freedom: 3822 Total (i.e. Null); 3816 Residual
## Null Deviance: 3213
## Residual Deviance: 3114 AIC: 3128
## log likelihood: -1557.07
## Nagelkerke R2: 0.04493401
## % pres/err predicted correctly: -941.134
## % of predictable range [ (model-null)/(1-null) ]: 0.02593565
## *****
## model index: 5
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept) stimlen log_freq pos stimlen:log_freq log_fr
## 3.24965 -0.17498 0.43038 -0.03639 -0.04515 0
##
## Degrees of Freedom: 3822 Total (i.e. Null); 3817 Residual
## Null Deviance: 3213
## Residual Deviance: 3117 AIC: 3129

```

```

## log likelihood: -1558.391
## Nagelkerke R2: 0.04374948
## % pres/err predicted correctly: -943.0359
## % of predictable range [ (model-null)/(1-null) ]: 0.02396928
## *****
## model index: 1
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept) stimlen log_freq stimlen:log_freq
## 3.25083 -0.19353 0.43018 -0.03956
##
## Degrees of Freedom: 3822 Total (i.e. Null); 3819 Residual
## Null Deviance: 3213
## Residual Deviance: 3121 AIC: 3129
## log likelihood: -1560.624
## Nagelkerke R2: 0.0417446
## % pres/err predicted correctly: -944.3213
## % of predictable range [ (model-null)/(1-null) ]: 0.02264031
## *****
## model index: 8
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept) stimlen pos log_freq
## 3.25872 -0.16998 -0.04254 0.11062
##
## Degrees of Freedom: 3822 Total (i.e. Null); 3819 Residual
## Null Deviance: 3213
## Residual Deviance: 3124 AIC: 3132
## log likelihood: -1561.993
## Nagelkerke R2: 0.04051365
## % pres/err predicted correctly: -944.2084
## % of predictable range [ (model-null)/(1-null) ]: 0.02275705
## *****
## model index: 2
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept) stimlen log_freq
## 3.2603 -0.1917 0.1102
##
## Degrees of Freedom: 3822 Total (i.e. Null); 3820 Residual
## Null Deviance: 3213
## Residual Deviance: 3128 AIC: 3134
## log likelihood: -1563.818
## Nagelkerke R2: 0.03887218
## % pres/err predicted correctly: -945.254

```

```

## % of predictable range [ (model-null)/(1-null) ]: 0.02167605
## *****
## model index: 7
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept)      stimlen          pos      log_freq pos:log_freq
## 3.2583633    -0.1697722    -0.0429046    0.1132280    -0.0006313
##
## Degrees of Freedom: 3822 Total (i.e. Null); 3818 Residual
## Null Deviance:      3213
## Residual Deviance: 3124 AIC: 3134
## log likelihood: -1561.992
## Nagelkerke R2: 0.04051515
## % pres/err predicted correctly: -944.2137
## % of predictable range [ (model-null)/(1-null) ]: 0.02275159
## *****
## model index: 21
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept)      stimlen      I(pos^2)          pos stimlen:I(pos^2)      stimlen:I(pos^2)
## 2.44802      -0.01839      -0.05409      0.54613      0.01020      -0.01020
##
## Degrees of Freedom: 3822 Total (i.e. Null); 3817 Residual
## Null Deviance:      3213
## Residual Deviance: 3131 AIC: 3143
## log likelihood: -1565.584
## Nagelkerke R2: 0.03728263
## % pres/err predicted correctly: -944.8066
## % of predictable range [ (model-null)/(1-null) ]: 0.02213854
## *****
## model index: 20
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept)      stimlen      I(pos^2)          pos
## 4.0295      -0.2204      0.0278      -0.2849
##
## Degrees of Freedom: 3822 Total (i.e. Null); 3819 Residual
## Null Deviance:      3213
## Residual Deviance: 3135 AIC: 3143
## log likelihood: -1567.7
## Nagelkerke R2: 0.03537525
## % pres/err predicted correctly: -946.4714
## % of predictable range [ (model-null)/(1-null) ]: 0.02041734
## *****
## model index: 17

```



```

##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##       data = PosDat)
##
## Coefficients:
## (Intercept)      stimlen          pos
##    3.50134    -0.20413    -0.04173
##
## Degrees of Freedom: 3822 Total (i.e. Null);  3820 Residual
## Null Deviance:      3213
## Residual Deviance: 3144  AIC: 3150
## log likelihood:  -1571.958
## Nagelkerke R2:  0.03153109
## % pres/err predicted correctly:  -948.9661
## % of predictable range [ (model-null)/(1-null) ]:  0.01783808
## *****
## model index:  15
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##       data = PosDat)
##
## Coefficients:
## (Intercept)      stimlen
##    3.5022    -0.2253
##
## Degrees of Freedom: 3822 Total (i.e. Null);  3821 Residual
## Null Deviance:      3213
## Residual Deviance: 3147  AIC: 3151
## log likelihood:  -1573.724
## Nagelkerke R2:  0.02993472
## % pres/err predicted correctly:  -949.9026
## % of predictable range [ (model-null)/(1-null) ]:  0.01686983
## *****
## model index:  18
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##       data = PosDat)
##
## Coefficients:
## (Intercept)      stimlen          pos  stimlen:pos
##    3.690349    -0.226539    -0.099956    0.006713
##
## Degrees of Freedom: 3822 Total (i.e. Null);  3819 Residual
## Null Deviance:      3213
## Residual Deviance: 3144  AIC: 3152
## log likelihood:  -1571.857
## Nagelkerke R2:  0.03162223
## % pres/err predicted correctly:  -948.9962
## % of predictable range [ (model-null)/(1-null) ]:  0.01780695
## *****
## model index:  4
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##       data = PosDat)

```

```

##
## Coefficients:
## (Intercept)          pos      log_freq
##      2.12500      -0.08968      0.14398
##
## Degrees of Freedom: 3822 Total (i.e. Null);  3820 Residual
## Null Deviance:      3213
## Residual Deviance: 3153  AIC: 3159
## log likelihood:  -1576.328
## Nagelkerke R2:  0.02757744
## % pres/err predicted correctly:  -950.8121
## % of predictable range [ (model-null)/(1-null) ]:  0.01592952
## *****
## model index:  9
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##      data = PosDat)
##
## Coefficients:
##      (Intercept)          I(pos^2)          pos      log_freq  I(pos^2):log_freq
##      2.35936      0.01447      -0.22345      0.06917      -0.00669
##
## Degrees of Freedom: 3822 Total (i.e. Null);  3817 Residual
## Null Deviance:      3213
## Residual Deviance: 3147  AIC: 3159
## log likelihood:  -1573.489
## Nagelkerke R2:  0.03014747
## % pres/err predicted correctly:  -949.2346
## % of predictable range [ (model-null)/(1-null) ]:  0.0175605
## *****
## model index:  3
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##      data = PosDat)
##
## Coefficients:
##      (Intercept)          pos      log_freq  pos:log_freq
##      2.137645      -0.093478      0.174117      -0.007396
##
## Degrees of Freedom: 3822 Total (i.e. Null);  3819 Residual
## Null Deviance:      3213
## Residual Deviance: 3152  AIC: 3160
## log likelihood:  -1576.101
## Nagelkerke R2:  0.02778263
## % pres/err predicted correctly:  -950.7969
## % of predictable range [ (model-null)/(1-null) ]:  0.0159453
## *****
## model index:  19
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##      data = PosDat)
##
## Coefficients:
##      (Intercept)          I(pos^2)          pos

```

```
##      2.38854      0.01558      -0.24228
##
## Degrees of Freedom: 3822 Total (i.e. Null);  3820 Residual
## Null Deviance:      3213
## Residual Deviance: 3186  AIC: 3192
## log likelihood:  -1592.976
## Nagelkerke R2:  0.01243148
## % pres/err predicted correctly:  -959.4523
## % of predictable range [ (model-null)/(1-null) ]:  0.006996463
## *****
## model index:  16
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##      data = PosDat)
##
## Coefficients:
## (Intercept)      pos
##      2.1546      -0.1037
##
## Degrees of Freedom: 3822 Total (i.e. Null);  3821 Residual
## Null Deviance:      3213
## Residual Deviance: 3189  AIC: 3193
## log likelihood:  -1594.357
## Nagelkerke R2:  0.01116939
## % pres/err predicted correctly:  -960.0668
## % of predictable range [ (model-null)/(1-null) ]:  0.006361135
## *****
## model index:  14
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##      data = PosDat)
##
## Coefficients:
## (Intercept)
##      1.746
##
## Degrees of Freedom: 3822 Total (i.e. Null);  3822 Residual
## Null Deviance:      3213
## Residual Deviance: 3213  AIC: 3215
## log likelihood:  -1606.533
## Nagelkerke R2:  1.952945e-16
## % pres/err predicted correctly:  -966.2195
## % of predictable range [ (model-null)/(1-null) ]:  0
## *****
```

```
BestFLPModel<-FLPRes$ModelResult[[1]]
BestFLPModelFormula<-FLPRes$Model[[1]]

FLPAICSummary<-data.frame(Model=FLPRes$Model,
                          AIC=FLPRes$AIC,row.names=FLPRes$Model)
FLPAICSummary$DeltaAIC<-FLPAICSummary$AIC-FLPAICSummary$AIC[1]
FLPAICSummary$AICexp<-exp(-0.5*FLPAICSummary$DeltaAIC)
FLPAICSummary$AICwt<-FLPAICSummary$AICexp/sum(FLPAICSummary$AICexp)
FLPAICSummary$NagR2<-FLPRes$NagR2
```

```

FLPAICSummary <- merge(FLPAICSummary,FLPRes$CoefficientValues,
                        by='row.names',sort=FALSE)
FLPAICSummary <- subset(FLPAICSummary, select = -c(Row.names))

write.csv(FLPAICSummary,paste0(TablesDir,CurPat,"_",
                               RunLabel,"_",CurTask,
                               "_freq_len_pos_model_summary.csv"),row.names = FALSE)

kable(FLPAICSummary)

```

Model	AIC Delta	AIC	AICw	NagR	Intercept	log_freq	stimlen	log_freq	pos	log_freq	I(pos^2)	log_freq	I(pos^2)	len:I(pos^2)
preserved ~ stimlen * log_freq + I(pos^2) + pos	3121.868	0.000000	0.000000	0.099	0.095	0.586	0.48	0.417	0.494	-	NA	NA	0.026	NA
					0.187	0.164	0.037	0.652	0.756	0.656				
preserved ~ stimlen * log_freq + (I(pos^2) + pos) * log_freq	3123.278	0.402	0.281	0.708	0.171	0.747	0.587	0.338	0.130	-	NA	0.048	0.002	NA
					0.191	0.053	0.042	0.363	0.260	0.841			0.003	0.982
preserved ~ stimlen + I(pos^2) + pos + log_freq	3125.674	0.805	0.401	0.923	0.754	0.378	0.408	0.110	0.340	-	NA	NA	0.027	NA
					0.186	0.341		0.284	0.582					
preserved ~ stimlen * log_freq + pos	3127.543	0.675	0.506	0.633	0.248	0.329	0.217	0.429	0.595	-	NA	NA	NA	NA
					0.171	0.950	0.039	0.448	0.422	0.356				
preserved ~ stimlen + (I(pos^2) + pos) * log_freq	3128.617	0.750	0.343	0.825	0.984	0.197	0.082	0.010	0.017	-	0.055	0.478	0.025	NA
					0.186	0.443		0.262	0.693			0.005	0.950	
preserved ~ stimlen * log_freq + pos *	3128.791	0.923	0.162	0.885	0.237	0.295	0.651	0.430	0.381	-	NA	0.010	0.146	NA
					0.174	0.976	0.045	0.145	0.363	0.862				
preserved ~ stimlen * log_freq	3129.248	0.382	0.402	0.644	0.404	0.174	0.250	0.430	0.175	NA	NA	NA	NA	NA
					0.193	0.507	0.039	0.560						
preserved ~ stimlen + pos + log_freq	3131.086	0.220	0.563	0.367	0.974	0.323	0.721	0.110	0.018	-	NA	NA	NA	NA
					0.169	0.769		0.042	0.540					
preserved ~ stimlen + log_freq	3133.636	0.770	0.770	0.608	0.820	0.302	0.110	0.211	0.211	NA	NA	NA	NA	NA
					0.191	0.700								
preserved ~ stimlen + pos * log_freq	3133.023	0.162	0.308	0.990	0.325	0.363	0.113	0.228	0.228	-	-	NA	NA	NA
					0.169	0.722		0.042	0.000	0.631				

[illegible]

```

PosDat$FLPFitted<-fitted(BestFLPModel)

HFDat <- PosDat[PosDat$freq_bin == "hf",]
LFDat <- PosDat[PosDat$freq_bin == "lf",]

HF_Plot <- plot_len_pos_obs_predicted(HFDat,paste0(CurPat," - ",RunLabel," - High frequency"),"FLPFitted")

## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
## Scale for y is already present. Adding another scale for y, which will replace the existing scale.
LF_Plot <- plot_len_pos_obs_predicted(LFDat,paste0(CurPat," - ",RunLabel," - Low frequency"),"FLPFitted")

## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
## Scale for y is already present. Adding another scale for y, which will replace the existing scale.

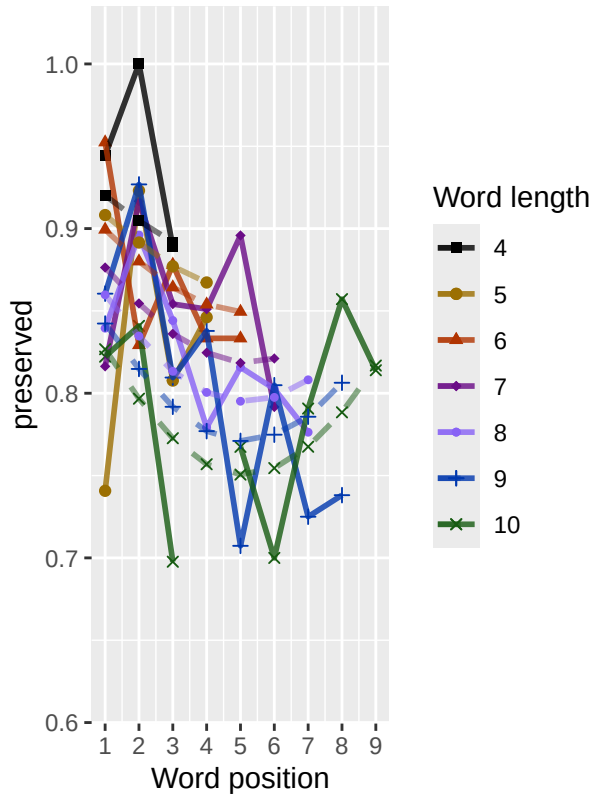
library(ggpubr)
Both_Plots <- ggarrange(LF_Plot,HF_Plot) # labels=c("LF","HF",ncol=2)

## Warning: Removed 1 row containing missing values or values outside the scale range (`geom_point()`).
## Removed 1 row containing missing values or values outside the scale range (`geom_point()`).

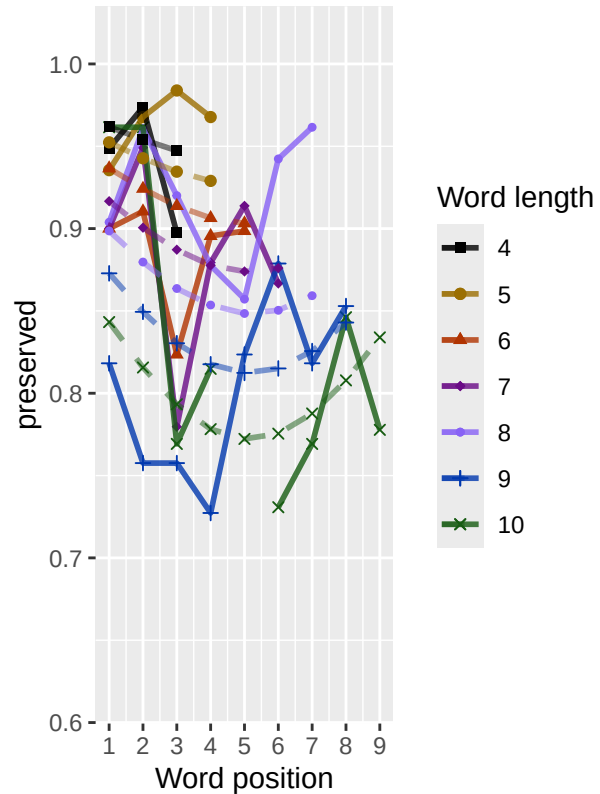
ggsave(paste0(FigDir,CurPat,"_",RunLabel,"_",
              CurTask,"_frequency_effect_length_pos_wfit.png"),
       device="png",unit="cm",width=30,height=11)
print(Both_Plots)

```

PV – no_frac – Low frequency



PV – no_frac – High frequency



```
# only main effects
MEModelEquations<-c(
  "preserved ~ CumPres",
  "preserved ~ CumErr",
  "preserved ~ (I(pos^2)+pos)",
  "preserved ~ pos",
  "preserved ~ stimlen",
  "preserved ~ 1"
)
MERes<-TestModels(MEModelEquations,PosDat)
```

```
## *****
## model index: 2
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept)      CumErr
##      2.0821      -0.6256
##
## Degrees of Freedom: 3822 Total (i.e. Null); 3821 Residual
## Null Deviance: 3213
## Residual Deviance: 3038 AIC: 3042
## log likelihood: -1518.96
## Nagelkerke R2: 0.07877042
## % pres/err predicted correctly: -910.5515
```

```

## % of predictable range [ (model-null)/(1-null) ]: 0.05755463
## *****
## model index: 5
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept)      stimlen
##      3.5022      -0.2253
##
## Degrees of Freedom: 3822 Total (i.e. Null); 3821 Residual
## Null Deviance:      3213
## Residual Deviance: 3147 AIC: 3151
## log likelihood: -1573.724
## Nagelkerke R2: 0.02993472
## % pres/err predicted correctly: -949.9026
## % of predictable range [ (model-null)/(1-null) ]: 0.01686983
## *****
## model index: 3
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept)      I(pos^2)      pos
##      2.38854      0.01558     -0.24228
##
## Degrees of Freedom: 3822 Total (i.e. Null); 3820 Residual
## Null Deviance:      3213
## Residual Deviance: 3186 AIC: 3192
## log likelihood: -1592.976
## Nagelkerke R2: 0.01243148
## % pres/err predicted correctly: -959.4523
## % of predictable range [ (model-null)/(1-null) ]: 0.006996463
## *****
## model index: 4
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept)      pos
##      2.1546      -0.1037
##
## Degrees of Freedom: 3822 Total (i.e. Null); 3821 Residual
## Null Deviance:      3213
## Residual Deviance: 3189 AIC: 3193
## log likelihood: -1594.357
## Nagelkerke R2: 0.01116939
## % pres/err predicted correctly: -960.0668
## % of predictable range [ (model-null)/(1-null) ]: 0.006361135
## *****
## model index: 1

```



```

##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##       data = PosDat)
##
## Coefficients:
## (Intercept)      CumPres
##      1.58839      0.07075
##
## Degrees of Freedom: 3822 Total (i.e. Null);  3821 Residual
## Null Deviance:      3213
## Residual Deviance: 3205  AIC: 3209
## log likelihood:  -1602.485
## Nagelkerke R2:  0.003720729
## % pres/err predicted correctly:  -964.2974
## % of predictable range [ (model-null)/(1-null) ]:  0.001987246
## *****
## model index:  6
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##       data = PosDat)
##
## Coefficients:
## (Intercept)
##      1.746
##
## Degrees of Freedom: 3822 Total (i.e. Null);  3822 Residual
## Null Deviance:      3213
## Residual Deviance: 3213  AIC: 3215
## log likelihood:  -1606.533
## Nagelkerke R2:  1.952945e-16
## % pres/err predicted correctly:  -966.2195
## % of predictable range [ (model-null)/(1-null) ]:  0
## *****

BestMEModel<-MERes$ModelResult[[ BestModelIndexL1 ]]
BestMEModelFormula<-MERes$Model[[BestModelIndexL1]]

MEAICSummary<-data.frame(Model=MERes$Model,
                          AIC=MERes$AIC,row.names=MERes$Model)
MEAICSummary$DeltaAIC<-MEAICSummary$AIC-MEAICSummary$AIC[1]
MEAICSummary$AICexp<-exp(-0.5*MEAICSummary$DeltaAIC)
MEAICSummary$AICwt<-MEAICSummary$AICexp/sum(MEAICSummary$AICexp)
MEAICSummary$NagR2<-MERes$NagR2

MEAICSummary <- merge(MEAICSummary,MERes$CoefficientValues,
                      by='row.names',sort=FALSE)
MEAICSummary <- subset(MEAICSummary, select = -c(Row.names))

write.csv(MEAICSummary,paste0(TablesDir,CurPat,"_",RunLabel,"_",
                             CurTask,"_main_effects_model_summary.csv"),
          row.names = FALSE)
kable(MEAICSummary)

```

Model	AIC	DeltaAIC	AICexp	AICwt	NagR2	(Intercept)	CumPres	CumErr	I(pos^2)	pos	stimlen
preserved ~ CumErr	3041.920	0.0000	1	1	0.0787702	0.082087	NA	-	NA	NA	NA
preserved ~ stimlen	3151.448	109.5273	0	0	0.0299343	0.502214	NA	0.6255733	NA	NA	-
preserved ~ (I(pos^2) + pos)	3191.952	150.0315	0	0	0.0124312	0.388538	NA	NA	0.0155804	-	NA
preserved ~ pos	3192.713	150.7930	0	0	0.0111692	0.154613	NA	NA	NA	0.2422753	NA
preserved ~ CumPres	3208.971	167.0502	0	0	0.0037207	0.588391	0.0707474	NA	NA	0.1037395	NA
preserved ~ 1	3215.065	173.1451	0	0	0.0000000	0.745826	NA	NA	NA	NA	NA

```

if(DoSimulations){
  BestMEModelFormulaRnd <- BestMEModelFormula
  if(grepl("CumPres",BestMEModelFormulaRnd)){
    BestMEModelFormulaRnd <- gsub("CumPres","RndCumPres",BestMEModelFormulaRnd)
  }else if(grepl("CumErr",BestMEModelFormulaRnd)){
    BestMEModelFormulaRnd <- gsub("CumErr","RndCumErr",BestMEModelFormulaRnd)
  }

  RndModelAIC<-numeric(length=RandomSamples)
  for(rindex in seq(1,RandomSamples)){
    # Shuffle cumulative values
    PosDat$RndCumPres <- ShuffleWithinWord(PosDat,"CumPres")
    PosDat$RndCumErr <- ShuffleWithinWord(PosDat,"CumErr")
    BestModelRnd <- glm(as.formula(BestMEModelFormulaRnd),
                        family="binomial",data=PosDat)
    RndModelAIC[rindex] <- BestModelRnd$aic
  }
  ModelNames<-c(paste0("***",BestMEModelFormula),
                 rep(BestMEModelFormulaRnd,RandomSamples))
  AICValues <- c(BestMEModel$aic,RndModelAIC)
  BestMEModelRndDF <- data.frame(Name=ModelNames,AIC=AICValues)
  BestMEModelRndDF <- BestMEModelRndDF %>% arrange(AIC)
  BestMEModelRndDF <- rbind(BestMEModelRndDF,
                            data.frame(Name=c("Random average"),
                                         AIC=c(mean(RndModelAIC))))
  BestMEModelRndDF <- rbind(BestMEModelRndDF,
                            data.frame(Name=c("Random SD"),
                                         AIC=c(sd(RndModelAIC))))

  write.csv(BestMEModelRndDF,
            paste0(TablesDir,CurPat,"_",RunLabel,"_",CurTask,
                  "_best_main_effects_model_with_random_cum_term.csv"),
            row.names = FALSE)
}

```

```

syll_component_summary <- PosDat %>%
  group_by(syll_component) %>% summarise(MeanPres = mean(preserved),
                                         N = n())
write.csv(syll_component_summary,paste0(TablesDir,CurPat,"_",
                                       RunLabel,"_",CurTask,

```

```

kable(syll_component_summary)                                     "_syllable_component_summary.csv"),row.names = FALSE)

```

syll_component	MeanPres	N
1	0.8693957	513
O	0.8288288	1776
P	0.8529412	34
S	0.8333333	222
V	0.8787167	1278

```

# main effects models for data without satellite positions

```

```

keep_components = c("0","V","1")
OV1Data <- PosDat[PosDat$syll_component %in% keep_components,]
OV1Data <- OV1Data %>% select(stim_number,
                             stimlen,stim,pos,
                             preserved,syll_component)
OV1Data$CumPres <- CalcCumPres(OV1Data)
OV1Data$CumErr <- CalcCumErrFromPreserved(OV1Data)

SimpSyllMEAICSummary <- EvaluateSubsetData(OV1Data,MEModelEquations)

```

```

## *****
## model index:  2
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##          data = PosDat)
##
## Coefficients:
## (Intercept)      CumErr
##      2.0834      -0.6577
##
## Degrees of Freedom: 3566 Total (i.e. Null);  3565 Residual
## Null Deviance:      2984
## Residual Deviance: 2820  AIC: 2824
## log likelihood:  -1410.027
## Nagelkerke R2:  0.07926822
## % pres/err predicted correctly:  -844.2667
## % of predictable range [ (model-null)/(1-null) ]:  0.05753614
## *****
## model index:  5
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##          data = PosDat)
##
## Coefficients:
## (Intercept)      stimlen
##      3.5535      -0.2305
##
## Degrees of Freedom: 3566 Total (i.e. Null);  3565 Residual
## Null Deviance:      2984
## Residual Deviance: 2921  AIC: 2925

```

```

## log likelihood: -1460.338
## Nagelkerke R2: 0.03105897
## % pres/err predicted correctly: -880.1151
## % of predictable range [ (model-null)/(1-null) ]: 0.0175655
## *****
## model index: 3
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept) I(pos^2) pos
## 2.37374 0.01442 -0.22979
##
## Degrees of Freedom: 3566 Total (i.e. Null); 3564 Residual
## Null Deviance: 2984
## Residual Deviance: 2960 AIC: 2966
## log likelihood: -1480.084
## Nagelkerke R2: 0.01176321
## % pres/err predicted correctly: -889.9697
## % of predictable range [ (model-null)/(1-null) ]: 0.006577776
## *****
## model index: 4
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept) pos
## 2.1569 -0.1015
##
## Degrees of Freedom: 3566 Total (i.e. Null); 3565 Residual
## Null Deviance: 2984
## Residual Deviance: 2962 AIC: 2966
## log likelihood: -1481.189
## Nagelkerke R2: 0.01067631
## % pres/err predicted correctly: -890.4298
## % of predictable range [ (model-null)/(1-null) ]: 0.006064776
## *****
## model index: 1
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept) CumPres
## 1.59393 0.07747
##
## Degrees of Freedom: 3566 Total (i.e. Null); 3565 Residual
## Null Deviance: 2984
## Residual Deviance: 2976 AIC: 2980
## log likelihood: -1487.896
## Nagelkerke R2: 0.004070033
## % pres/err predicted correctly: -893.9074

```

```
## % of predictable range [ (model-null)/(1-null) ]: 0.002187254
## *****
## model index: 6
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept)
## 1.755
##
## Degrees of Freedom: 3566 Total (i.e. Null); 3566 Residual
## Null Deviance: 2984
## Residual Deviance: 2984 AIC: 2986
## log likelihood: -1492.015
## Nagelkerke R2: -3.917483e-16
## % pres/err predicted correctly: -895.8691
## % of predictable range [ (model-null)/(1-null) ]: 0
## *****
```

```
write.csv(SimpSyllMEAICSummary,
          paste0(TablesDir, CurPat, "_", RunLabel, "_",
                CurTask,
                "_OV1_main_effects_model_summary.csv"),
          row.names = FALSE)
kable(SimpSyllMEAICSummary)
```

Model	AIC	DeltaAIC	AICexp	AICwt	NagR2	(Intercept)	CumPres	CumErr	I(pos^2)	pos	stimlen
preserved ~ CumErr	2824.054	0.0000	1	1	0.0792682	0.083439	NA	-	NA	NA	NA
preserved ~ stimlen	2924.675	100.6216	0	0	0.0310590	0.553480	NA	NA	NA	NA	-
preserved ~ (I(pos^2) + pos)	2966.167	142.1133	0	0	0.0117632	0.373736	NA	NA	0.0144164	-	NA
preserved ~ pos	2966.379	142.3248	0	0	0.0106763	0.156864	NA	NA	NA	-	NA
preserved ~ CumPres	2979.791	155.7374	0	0	0.0040700	0.593933	0.0774713	NA	NA	NA	NA
preserved ~ 1	2986.029	161.9757	0	0	0.0000000	0.754641	NA	NA	NA	NA	NA

```
# main effects models for data without satellite or coda positions
# (tradeoff -- takes away more complex segments, in addition to satellites, but
# also reduces data)
```

```
keep_components = c("0", "V")
OVData <- PosDat[PosDat$syll_component %in% keep_components,]
OVData <- OVData %>% select(stim_number,
                           stimlen, stim, pos,
                           preserved, syll_component)
OVData$CumPres <- CalcCumPres(OVData)
OVData$CumErr <- CalcCumErrFromPreserved(OVData)

SimpSyllMEAICSummary2 <- EvaluateSubsetData(OVData, MEModelEquations)
```

```

## *****
## model index: 2
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept)      CumErr
##      2.0334      -0.7147
##
## Degrees of Freedom: 3053 Total (i.e. Null); 3052 Residual
## Null Deviance:      2585
## Residual Deviance: 2457 AIC: 2461
## log likelihood: -1228.253
## Nagelkerke R2: 0.07216347
## % pres/err predicted correctly: -738.08
## % of predictable range [ (model-null)/(1-null) ]: 0.05249737
## *****
## model index: 5
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept)      stimlen
##      3.4949      -0.2266
##
## Degrees of Freedom: 3053 Total (i.e. Null); 3052 Residual
## Null Deviance:      2585
## Residual Deviance: 2531 AIC: 2535
## log likelihood: -1265.466
## Nagelkerke R2: 0.03074498
## % pres/err predicted correctly: -765.3903
## % of predictable range [ (model-null)/(1-null) ]: 0.01748549
## *****
## model index: 3
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept)      I(pos^2)      pos
##      2.36794      0.01459      -0.23617
##
## Degrees of Freedom: 3053 Total (i.e. Null); 3051 Residual
## Null Deviance:      2585
## Residual Deviance: 2561 AIC: 2567
## log likelihood: -1280.343
## Nagelkerke R2: 0.01390126
## % pres/err predicted correctly: -772.9283
## % of predictable range [ (model-null)/(1-null) ]: 0.007821695
## *****
## model index: 4
##

```

```

## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept)          pos
##      2.1550      -0.1069
##
## Degrees of Freedom: 3053 Total (i.e. Null); 3052 Residual
## Null Deviance:      2585
## Residual Deviance: 2563 AIC: 2567
## log likelihood: -1281.411
## Nagelkerke R2: 0.01268665
## % pres/err predicted correctly: -773.3608
## % of predictable range [ (model-null)/(1-null) ]: 0.007267314
## *****
## model index: 6
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept)
##      1.732
##
## Degrees of Freedom: 3053 Total (i.e. Null); 3053 Residual
## Null Deviance:      2585
## Residual Deviance: 2585 AIC: 2587
## log likelihood: -1292.514
## Nagelkerke R2: -3.888284e-16
## % pres/err predicted correctly: -779.0295
## % of predictable range [ (model-null)/(1-null) ]: 0
## *****
## model index: 1
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept)      CumPres
##      1.68336      0.02771
##
## Degrees of Freedom: 3053 Total (i.e. Null); 3052 Residual
## Null Deviance:      2585
## Residual Deviance: 2584 AIC: 2588
## log likelihood: -1292.168
## Nagelkerke R2: 0.000396694
## % pres/err predicted correctly: -778.8574
## % of predictable range [ (model-null)/(1-null) ]: 0.0002205851
## *****
write.csv(SimpSyllMEAICSummary2,
         paste0(TablesDir, CurPat, "_", RunLabel, "_", CurTask,
               "_OV_main_effects_model_summary.csv"), row.names = FALSE)
kable(SimpSyllMEAICSummary2)

```

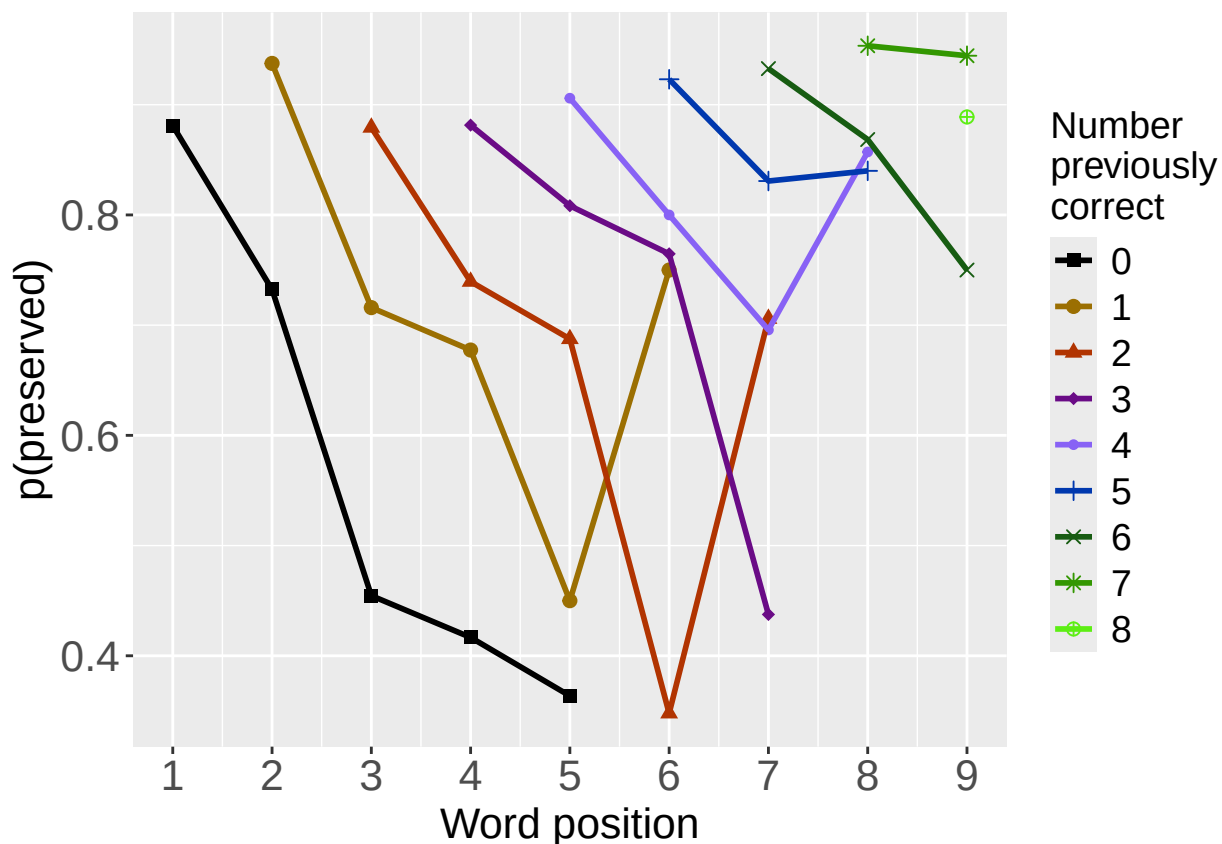
Model	AIC	DeltaAIC	AICexp	AICwt	NagR2	(Intercept)	CumPres	CumErr	I(pos^2)	pos	stimlen
preserved ~ CumErr	2460.500	0.00000	1	1	0.0721632	0.033384	NA	-	NA	NA	NA
preserved ~ stimlen	2534.937	4.42508	0	0	0.0307450	0.494890	NA	NA	NA	NA	-
preserved ~ (I(pos^2) + pos)	2566.686	106.18040	0	0	0.0139012	0.367943	NA	NA	0.0145939	-	NA
preserved ~ pos	2566.821	106.31491	0	0	0.0126867	0.154955	NA	NA	NA	-	NA
preserved ~ 1	2587.027	126.52122	0	0	0.0000000	0.732292	NA	NA	NA	NA	NA
preserved ~ CumPres	2588.335	127.82930	0	0	0.0003967	0.683358	0.0277057	NA	NA	NA	NA

```
# plot prev err and prev cor plots
```

```
PrevCorPlot<-PlotObsPreviousCorrect(PosDat,palette_values,shape_values)
```

```
## `summarise()` has grouped output by 'CumPres'. You can override using the `.groups` argument.
```

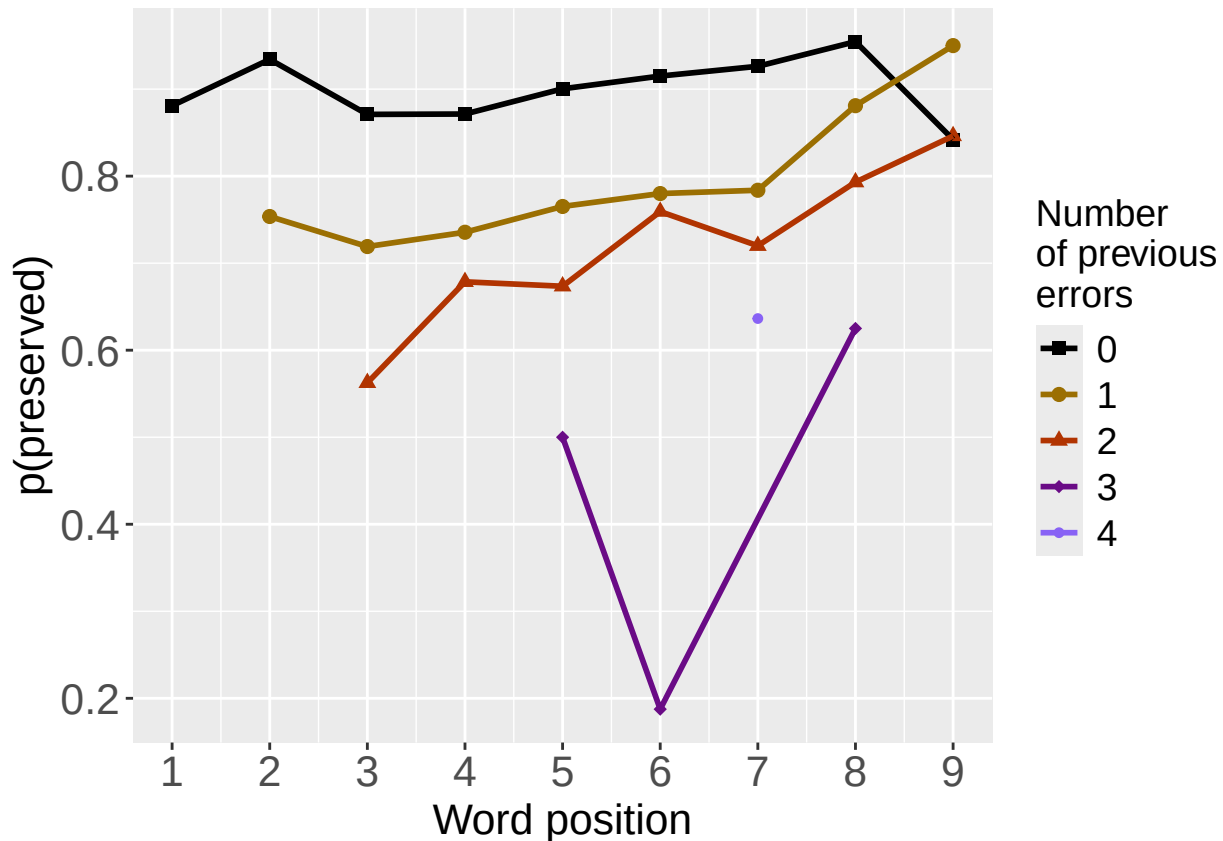
```
print(PrevCorPlot)
```



```
PrevErrPlot<-PlotObsPreviousError(PosDat,palette_values,shape_values)
```

```
## `summarise()` has grouped output by 'CumErr'. You can override using the `.groups` argument.
```

```
print(PrevErrPlot)
```

```

PlotName<-paste0(FigDir,CurPat,"_",RunLabel,"_",CurTask,"_PrevCorPlot_Obs")
ggsave(filename=paste0(PlotName,".tif"),plot=PrevCorPlot,device="tiff",compression = "lzw")

## Saving 6.5 x 4.5 in image

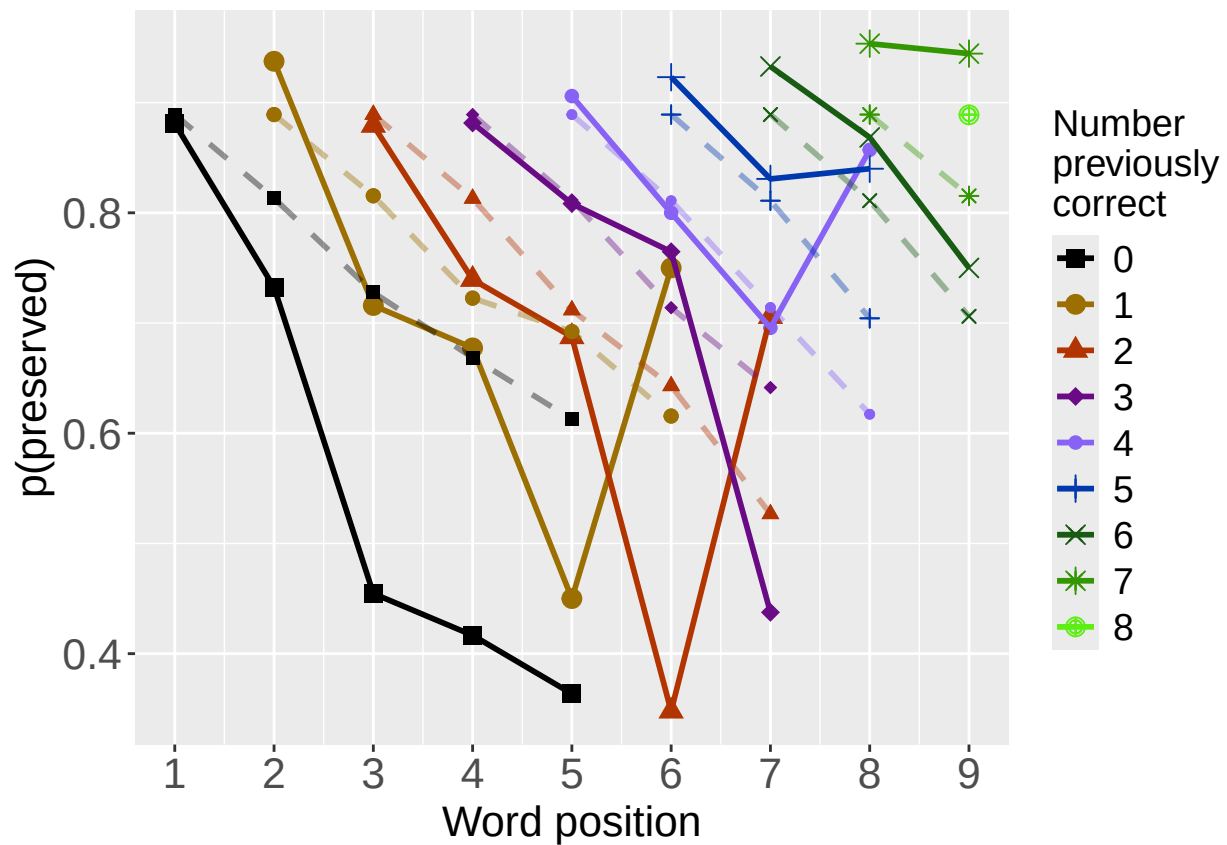
PlotName<-paste0(FigDir,CurPat,"_",RunLabel,"_",CurTask,"_PrevErrPlot_Obs")
ggsave(filename=paste0(PlotName,".tif"),plot=PrevErrPlot,device="tiff",compression = "lzw")

## Saving 6.5 x 4.5 in image
# plot prev err and prev cor with predicted values
MEModel<-MERes$ModelResult[[ BestModelIndexL1 ]]
PosDat$MEPred<-fitted(MEModel)

PrevCorPlot<-PlotObsPredPreviousCorrect(PosDat,"preserved","MEPred",palette_values,shape_values)

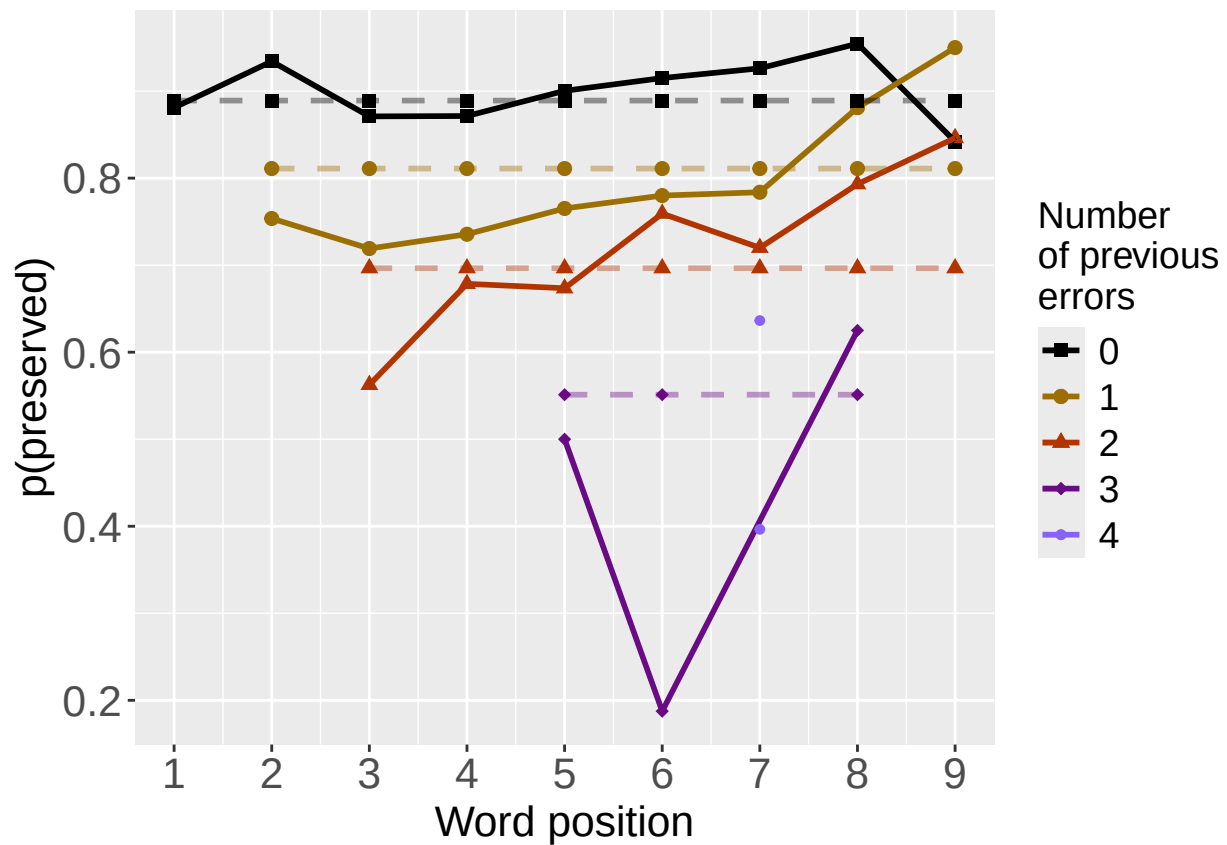
## `summarise()` has grouped output by 'CumPres'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'CumPres'. You can override using the `.groups` argument.
print(PrevCorPlot)

```



```
PrevErrPlot<-PlotObsPredPreviousError(PosDat,"preserved","MEPred",palette_values,shape_values)

## `summarise()` has grouped output by 'CumErr'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'CumErr'. You can override using the `.groups` argument.
print(PrevErrPlot)
```



```

PlotName<-paste0(FigDir,CurPat,"_",RunLabel,"_",
                  CurTask,"PrevCorPlot_ObsPredME")
ggsave(filename=paste0(PlotName,".tif"),plot=PrevCorPlot,device="tiff",compression = "lzw")

## Saving 6.5 x 4.5 in image

PlotName<-paste0(FigDir,CurPat,"_",RunLabel,"_",
                  CurTask,"PrevErrPlot_ObsPredME")
ggsave(filename=paste0(PlotName,".tif"),plot=PrevErrPlot,device="tiff",compression = "lzw")

## Saving 6.5 x 4.5 in image

```

```

CumAICSummary <- NULL

#####
# level 2 -- Add position squared (quadratic with position)
#####

# After establishing the primary variable, see about additions

BestMEFactor<-BestMEModelFormula
OtherFactor<-"I(pos^2) + pos"
OtherFactorName<-"quadratic_by_pos"

if(!grepl("pos",BestMEFactor,fixed = TRUE)){ # if best model does not include pos
  AICSummary<-TestLevel2Models(PosDat,BestMEFactor,OtherFactor,OtherFactorName)
  CumAICSummary <- AICSummary
  kable(AICSummary)
}

```

```

## *****
## model index: 2
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept)      CumErr      I(pos^2)         pos
##      2.37977      -0.72199      0.03408      -0.23473
##
## Degrees of Freedom: 3822 Total (i.e. Null);  3819 Residual
## Null Deviance:      3213
## Residual Deviance: 3021  AIC: 3029
## log likelihood:  -1510.668
## Nagelkerke R2:  0.08604404
## % pres/err predicted correctly:  -905.2878
## % of predictable range [ (model-null)/(1-null) ]:  0.0629967
## *****
## model index: 1
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",

```

```

##      data = PosDat)
##
## Coefficients:
## (Intercept)      CumErr
##      2.0821      -0.6256
##
## Degrees of Freedom: 3822 Total (i.e. Null);  3821 Residual
## Null Deviance:      3213
## Residual Deviance: 3038  AIC: 3042
## log likelihood:  -1518.96
## Nagelkerke R2:  0.07877042
## % pres/err predicted correctly:  -910.5515
## % of predictable range [ (model-null)/(1-null) ]:  0.05755463
## *****
## model index:  3
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##      data = PosDat)
##
## Coefficients:
## (Intercept)      I(pos^2)      pos
##      2.38854      0.01558      -0.24228
##
## Degrees of Freedom: 3822 Total (i.e. Null);  3820 Residual
## Null Deviance:      3213
## Residual Deviance: 3186  AIC: 3192
## log likelihood:  -1592.976
## Nagelkerke R2:  0.01243148
## % pres/err predicted correctly:  -959.4523
## % of predictable range [ (model-null)/(1-null) ]:  0.006996463
## *****

```

Model	AIC	DeltaAIC	AICexp	AICwt	NagR2	(Intercept)	CumErr	I(pos^2)	pos
preserved ~ CumErr + I(pos^2) + pos	3029.335	0.00000	1.0000000	0.9981532	0.0860440	2.379767	-0.7219920	0.0340849	-0.2347308
preserved ~ CumErr	3041.920	12.58492	0.0018502	0.0018468	0.0787704	2.082087	-0.6255733	NA	NA
preserved ~ I(pos^2) + pos	3191.952	162.61645	0.0000000	0.0000000	0.0124315	2.388538	NA	0.0155804	-0.2422753

```
#####
# level 2 -- Add length
#####

BestMEFactor<-BestMEModelFormula
OtherFactor<-"stimlen"

if(!grepl(OtherFactor,BestMEFactor,fixed = TRUE)){
  AICSummary<-TestLevel2Models(PosDat,BestMEFactor,OtherFactor)
  CumAICSummary <- ConcatAndAddMissingColumns(CumAICSummary,AICSummary)
  kable(AICSummary)
}

## *****
## model index: 2
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept) CumErr stimlen
## 3.0783 -0.5571 -0.1328
##
## Degrees of Freedom: 3822 Total (i.e. Null); 3820 Residual
## Null Deviance: 3213
## Residual Deviance: 3018 AIC: 3024
## log likelihood: -1509.064
## Nagelkerke R2: 0.08744678
## % pres/err predicted correctly: -906.7383
## % of predictable range [ (model-null)/(1-null) ]: 0.06149704
## *****
## model index: 1
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept) CumErr
## 2.0821 -0.6256
##
## Degrees of Freedom: 3822 Total (i.e. Null); 3821 Residual
## Null Deviance: 3213
## Residual Deviance: 3038 AIC: 3042
## log likelihood: -1518.96
## Nagelkerke R2: 0.07877042
## % pres/err predicted correctly: -910.5515
## % of predictable range [ (model-null)/(1-null) ]: 0.05755463
## *****
## model index: 3
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
```

```
## (Intercept)      stimlen
##      3.5022      -0.2253
##
## Degrees of Freedom: 3822 Total (i.e. Null);  3821 Residual
## Null Deviance:      3213
## Residual Deviance: 3147  AIC: 3151
## log likelihood:  -1573.724
## Nagelkerke R2:  0.02993472
## % pres/err predicted correctly:  -949.9026
## % of predictable range [ (model-null)/(1-null) ]:  0.01686983
## *****
```

Model	AIC	DeltaAIC	AICexp	AICwt	NagR2	(Intercept)	CumErr	stimlen
preserved ~ CumErr + stimlen	3024.129	0.00000	1.000000	0.9998631	0.0874468	3.078338	- 0.5570736	- 0.1328068
preserved ~ CumErr	3041.920	17.79167	0.000137	0.0001369	0.0787704	2.082087	- 0.6255733	NA
preserved ~ stimlen	3151.448	127.31899	0.000000	0.0000000	0.0299347	3.502214	NA	- 0.2253417

```
#####
# level 2 -- add cumulative preserved
#####

BestMEFactor<-BestMEModelFormula
OtherFactor<-"CumPres"

if(!grepl(OtherFactor,BestMEFactor,fixed = TRUE)){
  AICSummary<-TestLevel2Models(PosDat,BestMEFactor,OtherFactor)
  CumAICSummary <- ConcatAndAddMissingColumns(CumAICSummary,AICSummary)
  kable(AICSummary)
}
```

```
## *****
## model index:  2
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##      data = PosDat)
##
## Coefficients:
## (Intercept)      CumErr      CumPres
##      1.87037      -0.64014      0.09807
##
## Degrees of Freedom: 3822 Total (i.e. Null);  3820 Residual
## Null Deviance:      3213
## Residual Deviance: 3024  AIC: 3030
## log likelihood:  -1511.973
## Nagelkerke R2:  0.08490143
## % pres/err predicted correctly:  -905.9614
## % of predictable range [ (model-null)/(1-null) ]:  0.06230027
## *****
## model index:  1
##
```

```
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##       data = PosDat)
##
## Coefficients:
## (Intercept)      CumErr
##      2.0821      -0.6256
##
## Degrees of Freedom: 3822 Total (i.e. Null);  3821 Residual
## Null Deviance:      3213
## Residual Deviance: 3038  AIC: 3042
## log likelihood:  -1518.96
## Nagelkerke R2:  0.07877042
## % pres/err predicted correctly:  -910.5515
## % of predictable range [ (model-null)/(1-null) ]:  0.05755463
## *****
## model index:  3
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##       data = PosDat)
##
## Coefficients:
## (Intercept)      CumPres
##      1.58839      0.07075
##
## Degrees of Freedom: 3822 Total (i.e. Null);  3821 Residual
## Null Deviance:      3213
## Residual Deviance: 3205  AIC: 3209
## log likelihood:  -1602.485
## Nagelkerke R2:  0.003720729
## % pres/err predicted correctly:  -964.2974
## % of predictable range [ (model-null)/(1-null) ]:  0.001987246
## *****
```

Model	AIC	DeltaAIC	AICexp	AICwt	NagR2	(Intercept)	CumErr	CumPres
preserved ~ CumErr + CumPres	3029.946	0.00000	1.0000000	0.9974961	0.0849014	1.870373	- 0.6401445	0.0980715
preserved ~ CumErr	3041.920	11.97482	0.0025102	0.0025039	0.0787704	2.082087	- 0.6255733	NA
preserved ~ CumPres	3208.971	179.02499	0.0000000	0.0000000	0.0037207	1.588391	NA	0.0707474

```
#####
# level 2 -- Add linear position (NOT quadratic)
#####

BestMEFactor<-BestMEModelFormula
OtherFactor<-"pos"

if(!grepl(OtherFactor,BestMEFactor,fixed = TRUE)){
  AICSummary<-TestLevel2Models(PosDat,BestMEFactor,OtherFactor)
  CumAICSummary <- ConcatAndAddMissingColumns(CumAICSummary,AICSummary)
  kable(AICSummary)
}
```



```

## *****
## model index: 2
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept)      CumErr      pos
##      1.89918      -0.69472      0.05636
##
## Degrees of Freedom: 3822 Total (i.e. Null); 3820 Residual
## Null Deviance:      3213
## Residual Deviance: 3033 AIC: 3039
## log likelihood: -1516.579
## Nagelkerke R2: 0.08086268
## % pres/err predicted correctly: -908.8303
## % of predictable range [ (model-null)/(1-null) ]: 0.05933414
## *****
## model index: 1
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept)      CumErr
##      2.0821      -0.6256
##
## Degrees of Freedom: 3822 Total (i.e. Null); 3821 Residual
## Null Deviance:      3213
## Residual Deviance: 3038 AIC: 3042
## log likelihood: -1518.96
## Nagelkerke R2: 0.07877042
## % pres/err predicted correctly: -910.5515
## % of predictable range [ (model-null)/(1-null) ]: 0.05755463
## *****
## model index: 3
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept)      pos
##      2.1546      -0.1037
##
## Degrees of Freedom: 3822 Total (i.e. Null); 3821 Residual
## Null Deviance:      3213
## Residual Deviance: 3189 AIC: 3193
## log likelihood: -1594.357
## Nagelkerke R2: 0.01116939
## % pres/err predicted correctly: -960.0668
## % of predictable range [ (model-null)/(1-null) ]: 0.006361135
## *****

```

Model	AIC	DeltaAIC	AICexp	AICwt	NagR2	(Intercept)	CumErr	pos
preserved ~ CumErr + pos	3039.157	0.00000	1.0000000	0.7992543	0.0808627	1.899184	- 0.6947234	0.0563603
preserved ~ CumErr	3041.920	2.76328	0.2511663	0.2007457	0.0787704	2.082087	- 0.6255733	NA
preserved ~ pos	3192.713	153.55626	0.0000000	0.0000000	0.0111694	2.154613	NA	- 0.1037395

```
CumAICSummary <- CumAICSummary %>% arrange(AIC)
BestModelFormulaL2 <- CumAICSummary$Model[BestModelIndexL2]
write.csv(CumAICSummary, paste0(TablesDir, CurPat, "_",
                                RunLabel, "_", CurTask,
                                "_main_effects_plus_one_model_summary.csv"), row.names = FALSE)
kable(CumAICSummary)
```

Model	AIC	DeltaAIC	AICexp	AICwt	NagR2	(Intercept)	CumErr	I(pos^2)	pos	stimlen	CumPres
preserved ~ CumErr + stimlen	3024.120	0.00000	1.0000000	0.9998631	0.0874468	0.878338	- 0.5570736	NA	NA	- 0.1328068	NA
preserved ~ CumErr + I(pos^2) + pos	3029.336	0.00000	1.0000000	0.9981532	0.0860442	0.379767	- 0.7219920	0.0340849	- 0.2347308	NA	NA
preserved ~ CumErr + CumPres	3029.946	0.00000	1.0000000	0.9974961	0.0849014	0.870373	- 0.6401445	NA	NA	NA	0.0980715
preserved ~ CumErr + pos	3039.157	0.00000	1.0000000	0.7992543	0.0808627	1.899184	- 0.6947234	NA	0.0563603	NA	NA
preserved ~ CumErr	3041.920	2.58492	0.0018502	0.0018468	0.0787704	2.082087	- 0.6255733	NA	NA	NA	NA
preserved ~ CumErr	3041.920	7.79167	0.0001370	0.0001369	0.0787704	2.082087	- 0.6255733	NA	NA	NA	NA
preserved ~ CumErr	3041.920	1.97482	0.0025102	0.0025039	0.0787704	2.082087	- 0.6255733	NA	NA	NA	NA
preserved ~ CumErr	3041.920	2.76328	0.2511663	0.2007457	0.0787704	2.082087	- 0.6255733	NA	NA	NA	NA
preserved ~ stimlen	3151.443	27.31899	0.0000000	0.0000000	0.0299347	5.02214	NA	NA	NA	- 0.2253417	NA
preserved ~ I(pos^2) + pos	3191.952	62.61645	0.0000000	0.0000000	0.0124315	3.88538	NA	0.0155804	- 0.2422753	NA	NA
preserved ~ pos	3192.713	53.55626	0.0000000	0.0000000	0.0111694	2.154613	NA	NA	- 0.1037395	NA	NA
preserved ~ CumPres	3208.971	79.02499	0.0000000	0.0000000	0.0037207	5.88391	NA	NA	NA	NA	0.0707474

explore influence of frequency and length

```
if(grepl("stimlen", BestModelFormulaL2)){ # if length is already in the formula
  Level3ModelEquations<-c(
    BestModelFormulaL2, # best model from level 2
    "preserved ~ 1", # null model
    paste0(BestModelFormulaL2, " + log_freq")
  )
}
```

```

}else if(grepl("log_freq",BestModelFormulaL2)){ # if log_freq is already in the formula
  Level3ModelEquations<-c(
    BestModelFormulaL2, # best model from level 2
    "preserved ~ 1", # null model
    paste0(BestModelFormulaL2," + stimlen")
  )
}else{
  Level3ModelEquations<-c(
    BestModelFormulaL2, # best model from level 2
    "preserved ~ 1", # null model
    paste0(BestModelFormulaL2," + log_freq"),
    paste0(BestModelFormulaL2," + stimlen"),
    paste0(BestModelFormulaL2," + stimlen + log_freq")
  )
}
}

Level3Res<-TestModels(Level3ModelEquations,PosDat)

```

```

## *****
## model index: 3
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept)      CumErr      stimlen      log_freq
##      2.87337      -0.54797      -0.10474      0.09976
##
## Degrees of Freedom: 3822 Total (i.e. Null); 3819 Residual
## Null Deviance:      3213
## Residual Deviance: 3003 AIC: 3011
## log likelihood: -1501.277
## Nagelkerke R2: 0.09424323
## % pres/err predicted correctly: -903.2866
## % of predictable range [ (model-null)/(1-null) ]: 0.06506576
## *****
## model index: 1
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept)      CumErr      stimlen
##      3.0783      -0.5571      -0.1328
##
## Degrees of Freedom: 3822 Total (i.e. Null); 3820 Residual
## Null Deviance:      3213
## Residual Deviance: 3018 AIC: 3024
## log likelihood: -1509.064
## Nagelkerke R2: 0.08744678
## % pres/err predicted correctly: -906.7383
## % of predictable range [ (model-null)/(1-null) ]: 0.06149704
## *****
## model index: 2

```

```
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##       data = PosDat)
##
## Coefficients:
## (Intercept)
##      1.746
##
## Degrees of Freedom: 3822 Total (i.e. Null);  3822 Residual
## Null Deviance:      3213
## Residual Deviance: 3213  AIC: 3215
## log likelihood:  -1606.533
## Nagelkerke R2:   1.952945e-16
## % pres/err predicted correctly:  -966.2195
## % of predictable range [ (model-null)/(1-null) ]:  0
## *****
```

```
BestModelL3<-Level3Res$ModelResult[[ BestModelIndexL3 ]]
BestModelFormulaL3<-Level3Res$Model[[BestModelIndexL3]]

AICSummary<-data.frame(Model=Level3Res$Model,
                        AIC=Level3Res$AIC,
                        row.names = Level3Res$Model)
AICSummary$DeltaAIC<-AICSummary$AIC-AICSummary$AIC[1]
AICSummary$AICexp<-exp(-0.5*AICSummary$DeltaAIC)
AICSummary$AICwt<-AICSummary$AICexp/sum(AICSummary$AICexp)
AICSummary$NagR2<-Level3Res$NagR2

AICSummary <- merge(AICSummary,Level3Res$CoefficientValues,
                    by='row.names',sort=FALSE)
AICSummary <- subset(AICSummary, select = -c(1:10))

write.csv(AICSummary,paste0(TablesDir,CurPat,"_",RunLabel,"_",
                          CurTask,"_main_effects_plus_one_len_freq_summary.csv"),
          row.names = FALSE)

kable(AICSummary)
```

Model	AIC	DeltaAIC	AICexp	AICwt	NagR2	(Intercept)	CumErr	stimlen	log_freq
preserved ~ CumErr + stimlen + log_freq	3010.5530	0.00000	1.000000	0.998873	0.094243	2.873365	-	-	0.0997553
preserved ~ CumErr + stimlen	3024.1291	3.57528	0.001127	0.001126	0.087446	3.078338	-	-	NA
preserved ~ 1	3215.0652	204.5120	0.000000	0.000000	0.000000	1.745826	NA	NA	NA

```
# BestModelIndex is set by the parameter file and is used to choose
# a model that is essentially equivalent to the lowest AIC model, but
# simpler (and with a slightly higher AIC value, so not in first position)
BestModelL3<-Level3Res$ModelResult[[ BestModelIndexL3 ]]
BestModelFormulaL3<-Level3Res$Model[[BestModelIndexL3]]

BestModel<-BestModelL3
BestModelDeletions<-arrange(dropterm(BestModel),desc(AIC))
```

```

# order by terms that increase AIC the most when they are the one dropped
BestModelDeletions

## Single term deletions
##
## Model:
## preserved ~ CumErr + stimlen + log_freq
##      Df Deviance    AIC
## CumErr   1   3127.6 3133.6
## log_freq  1   3018.1 3024.1
## stimlen   1   3014.0 3020.0
## <none>      3002.6 3010.6

#####
# Single deletions from best model
#####

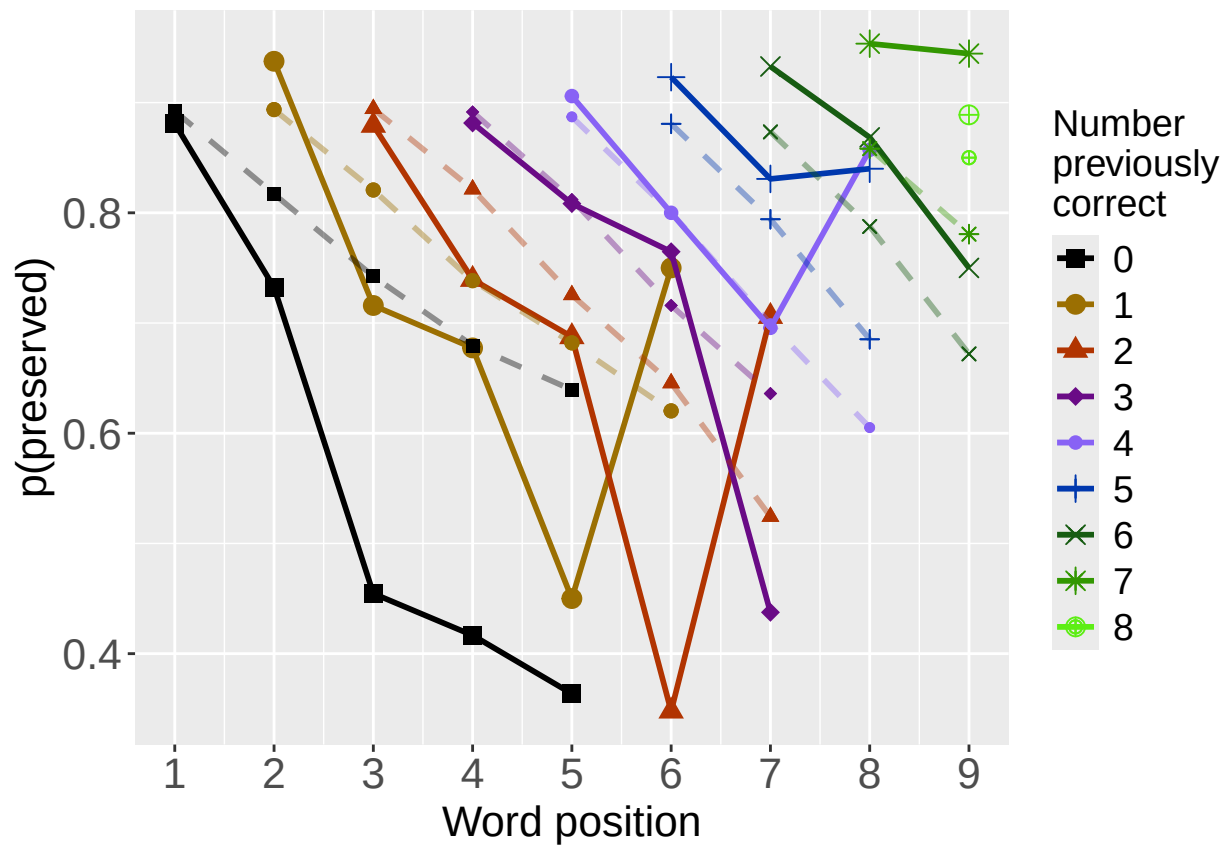
write.csv(BestModelDeletions,paste0(TablesDir,CurPat,"_",
                                   RunLabel,"_",CurTask,
                                   "_best_model_single_term_deletions.csv"),
          row.names = TRUE)

# plot prev err and prev cor with predicted values
PosDat$OAPred<-fitted(BestModel)

PrevCorPlot<-PlotObsPredPreviousCorrect(PosDat,"preserved","OAPred",palette_values,shape_values)

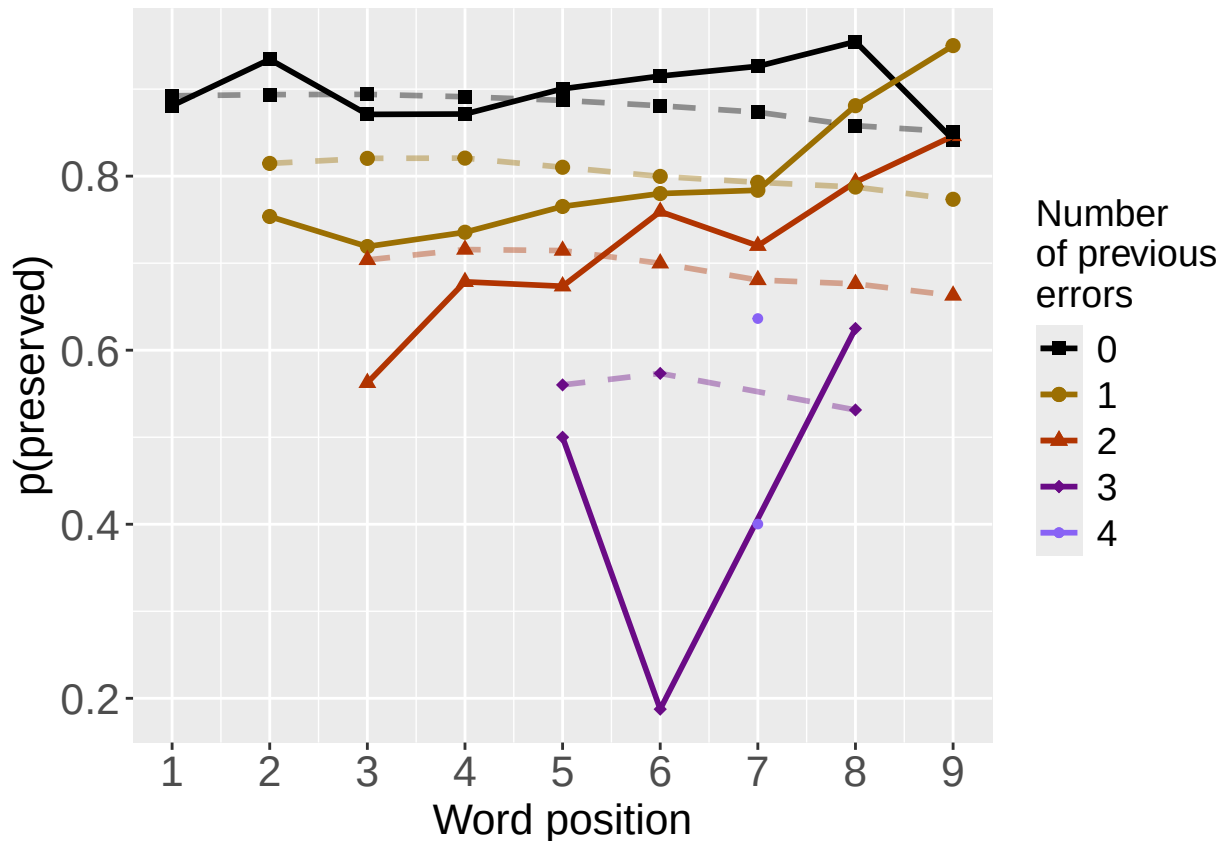
## `summarise()` has grouped output by 'CumPres'. You can override using the `groups` argument.
## `summarise()` has grouped output by 'CumPres'. You can override using the `groups` argument.
print(PrevCorPlot)

```



```
PrevErrPlot<-PlotObsPredPreviousError(PosDat,"preserved","OAPred",palette_values,shape_values)

## `summarise()` has grouped output by 'CumErr'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'CumErr'. You can override using the `.groups` argument.
print(PrevErrPlot)
```



```

PlotName<-paste0(FigDir,CurPat,"_",
  RunLabel,"_",CurTask,
  "_ObsPred_BestFullModel")
ggsave(filename=paste(PlotName,"_prev_correct.tif",sep=""),plot=PrevCorPlot,device="tiff",compression =

## Saving 6.5 x 4.5 in image
ggsave(filename=paste(PlotName,"_prev_error.tif",sep=""),plot=PrevErrPlot,device="tiff",compression = "

## Saving 6.5 x 4.5 in image
if(DoSimulations){
  BestModelFormulaL3Rnd <- BestModelFormulaL3
  if(grepl("CumPres",BestModelFormulaL3Rnd)){
    BestModelFormulaL3Rnd <- gsub("CumPres","RndCumPres",BestModelFormulaL3Rnd)
  }
  if(grepl("CumErr",BestModelFormulaL3Rnd)){
    BestModelFormulaL3Rnd <- gsub("CumErr","RndCumErr",BestModelFormulaL3Rnd)
  }

  RndModelAIC<-numeric(length=RandomSamples)
  for(rindex in seq(1,RandomSamples)){
    # Shuffle cumulative values
    PosDat$RndCumPres <- ShuffleWithinWord(PosDat,"CumPres")
    PosDat$RndCumErr <- ShuffleWithinWord(PosDat,"CumErr")
    BestModelRnd <- glm(as.formula(BestModelFormulaL3Rnd),
      family="binomial",data=PosDat)
    RndModelAIC[rindex] <- BestModelRnd$aic
  }
}

```

```

}
ModelNames<-c(paste0("***",BestModelFormulaL3),
              rep(BestModelFormulaL3Rnd,RandomSamples))
AICValues <- c(BestModelL3$aic,RndModelAIC)
BestModelRndDF <- data.frame(Name=ModelNames,AIC=AICValues)
BestModelRndDF <- BestModelRndDF %>% arrange(AIC)
BestModelRndDF <- rbind(BestModelRndDF,
                        data.frame(Name=c("Random average"),
                                   AIC=c(mean(RndModelAIC))))
BestModelRndDF <- rbind(BestModelRndDF,
                        data.frame(Name=c("Random SD"),
                                   AIC=c(sd(RndModelAIC))))
write.csv(BestModelRndDF,paste0(TablesDir,CurPat,"_",RunLabel,"_",CurTask,
                                "_best_model_with_random_cum_term.csv"),
          row.names = FALSE)
}

# plot influence factors
num_models <- nrow(BestModelDeletions) - 1
# minus 1 because last
# model is always <none> and we don't want to include that

if(num_models > 4){
  last_model_num <- 4
}else{
  last_model_num <- num_models
}

FinalModelSet<-ModelSetFromDeletions(BestModelFormulaL3,
                                     BestModelDeletions,
                                     last_model_num)

PlotName<-paste0(FigDir,CurPat,"_",RunLabel,"_",CurTask,"_FactorPlots")

FactorPlot<-PlotModelSetWithFreq(PosDat,FinalModelSet,
                                 palette_values,FinalModelSet,PptID=CurPat)

## *****
## model index: 1
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##          data = PosDat)
##
## Coefficients:
## (Intercept)      CumErr
##      2.0821      -0.6256
##
## Degrees of Freedom: 3822 Total (i.e. Null); 3821 Residual
## Null Deviance:      3213
## Residual Deviance: 3038 AIC: 3042
## log likelihood: -1518.96
## Nagelkerke R2: 0.07877042
## % pres/err predicted correctly: -910.5515
## % of predictable range [ (model-null)/(1-null) ]: 0.05755463

```



```

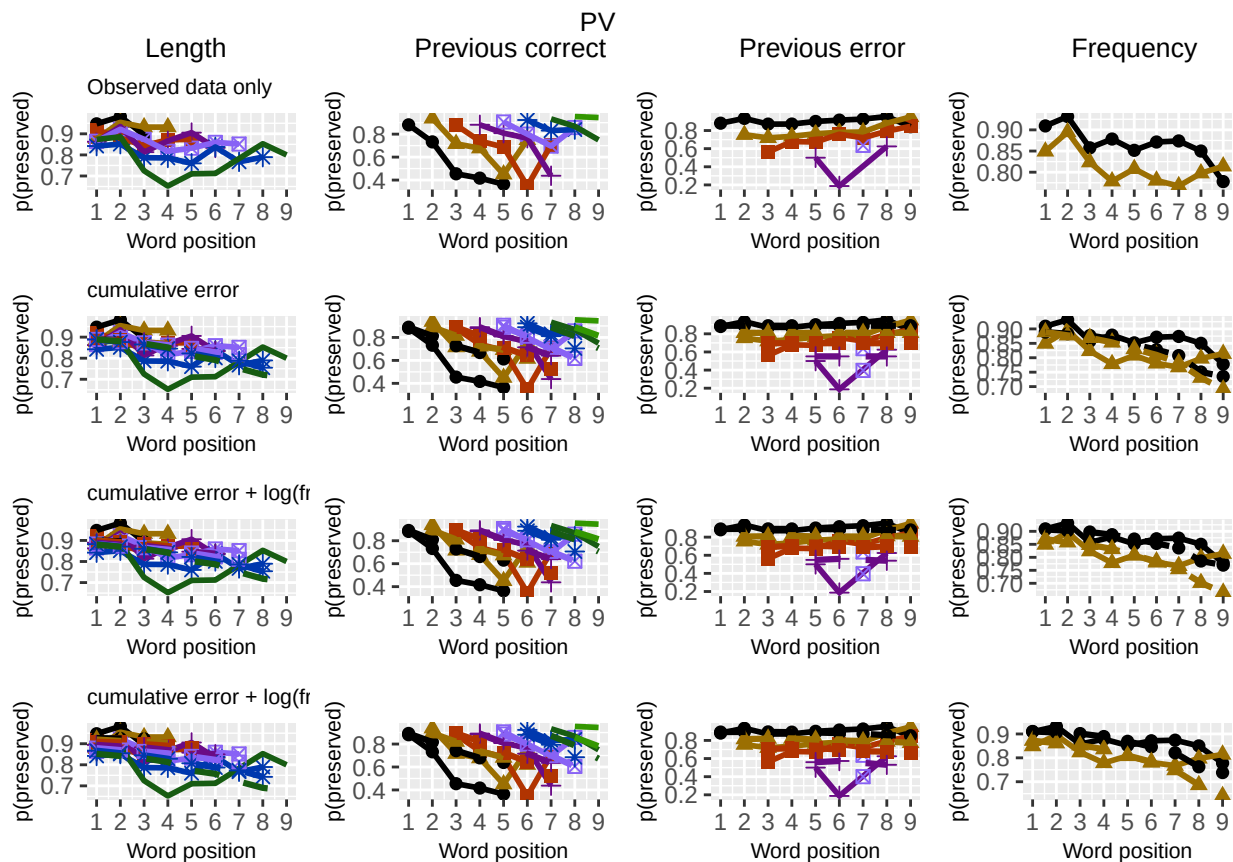
## *****
## model index: 2
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept)      CumErr      log_freq
##      2.0872      -0.5957      0.1201
##
## Degrees of Freedom: 3822 Total (i.e. Null); 3820 Residual
## Null Deviance:      3213
## Residual Deviance: 3014 AIC: 3020
## log likelihood: -1507.006
## Nagelkerke R2: 0.08924583
## % pres/err predicted correctly: -905.2772
## % of predictable range [ (model-null)/(1-null) ]: 0.06300771
## *****
## model index: 3
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept)      CumErr      log_freq      stimlen
##      2.87337      -0.54797      0.09976      -0.10474
##
## Degrees of Freedom: 3822 Total (i.e. Null); 3819 Residual
## Null Deviance:      3213
## Residual Deviance: 3003 AIC: 3011
## log likelihood: -1501.277
## Nagelkerke R2: 0.09424323
## % pres/err predicted correctly: -903.2866
## % of predictable range [ (model-null)/(1-null) ]: 0.06506576
## *****
## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'CumPres'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'CumErr'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'freq_bin'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'CumPres'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'CumErr'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'freq_bin'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'CumPres'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'CumErr'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'freq_bin'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'CumPres'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'CumErr'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'freq_bin'. You can override using the `.groups` argument.
## Warning: The shape palette can deal with a maximum of 6 discrete values because more than 6 becomes
## i you have requested 7 values. Consider specifying shapes manually if you need that many have them.

```

```

## Warning: Removed 9 rows containing missing values or values outside the scale range (`geom_point()`).
## Warning: The shape palette can deal with a maximum of 6 discrete values because more than 6 becomes
## i you have requested 9 values. Consider specifying shapes manually if you need that many have them.
## Warning: Removed 6 rows containing missing values or values outside the scale range (`geom_point()`).
## Warning: The shape palette can deal with a maximum of 6 discrete values because more than 6 becomes
## i you have requested 7 values. Consider specifying shapes manually if you need that many have them.
## Warning: Removed 9 rows containing missing values or values outside the scale range (`geom_point()`).
## Removed 9 rows containing missing values or values outside the scale range (`geom_point()`).
## Warning: The shape palette can deal with a maximum of 6 discrete values because more than 6 becomes
## i you have requested 9 values. Consider specifying shapes manually if you need that many have them.
## Warning: Removed 6 rows containing missing values or values outside the scale range (`geom_point()`).
## Removed 6 rows containing missing values or values outside the scale range (`geom_point()`).
## Warning: The shape palette can deal with a maximum of 6 discrete values because more than 6 becomes
## i you have requested 7 values. Consider specifying shapes manually if you need that many have them.
## Warning: Removed 9 rows containing missing values or values outside the scale range (`geom_point()`).
## Removed 9 rows containing missing values or values outside the scale range (`geom_point()`).
## Warning: The shape palette can deal with a maximum of 6 discrete values because more than 6 becomes
## i you have requested 9 values. Consider specifying shapes manually if you need that many have them.
## Warning: Removed 6 rows containing missing values or values outside the scale range (`geom_point()`).
## Removed 6 rows containing missing values or values outside the scale range (`geom_point()`).
## Warning: The shape palette can deal with a maximum of 6 discrete values because more than 6 becomes
## i you have requested 7 values. Consider specifying shapes manually if you need that many have them.
## Warning: Removed 9 rows containing missing values or values outside the scale range (`geom_point()`).
## Removed 9 rows containing missing values or values outside the scale range (`geom_point()`).
## Warning: The shape palette can deal with a maximum of 6 discrete values because more than 6 becomes
## i you have requested 9 values. Consider specifying shapes manually if you need that many have them.
## Warning: Removed 6 rows containing missing values or values outside the scale range (`geom_point()`).
## Removed 6 rows containing missing values or values outside the scale range (`geom_point()`).
ggsave(paste0(PlotName, ".tif"), plot=FactorPlot, width = 360, height=400, units="mm", device="tiff", compress=
# use \blandscape and \elandscape to make markdown plots landscape if needed
FactorPlot

```



```

DA.Result<-dominanceAnalysis(BestModel)
DAContributionAverage<-ConvertDominanceResultToTable(DA.Result)
write.csv(DAContributionAverage,paste0(TablesDir,CurPat,"_",RunLabel,"_",
                                     CurTask,"_dominance_analysis_table.csv"),
          row.names = TRUE)
kable(DAContributionAverage)

```

	CumErr	stimlen	log_freq
McFadden	0.0459796	0.0112540	0.0082839
SquaredCorrelation	0.0374853	0.0092857	0.0068050
Nagelkerke	0.0374853	0.0092857	0.0068050
Estrella	0.0388736	0.0094927	0.0069937

	deviance	deviance_explained
CumErr + log_freq + stimlen	3002.553	210.5120
CumErr + log_freq	3014.012	199.0534
CumErr	3037.920	175.1451
null	3213.065	0.0000

	percent_explained	percent_of_explained_deviance	increment_in_explained
CumErr + log_freq + stimlen	6.551750	100.00000	5.443207
CumErr + log_freq	6.195125	94.55679	11.357233
CumErr	5.451028	83.19956	83.199560
null	0.000000	NA	0.000000

```
deviance_differences_df<-GetDevianceSet(FinalModelSet,PosDat)
write.csv(deviance_differences_df,paste0(TablesDir,CurPat,"_",RunLabel,"_",
                                         CurTask,"_deviance_differences_table.csv"),
          row.names = FALSE)
deviance_differences_df
```

```
##                                model deviance deviance_explained percent_explained percent_of_explained_deviance
## CumErr + log_freq + stimlen CumErr + log_freq + stimlen 3002.553          210.5120          6.551751          100.00000
## CumErr + log_freq          CumErr + log_freq 3014.012          199.0534          6.195125          94.55679
## CumErr                    CumErr 3037.920          175.1451          5.451028          83.19956
## null                      null 3213.065          0.0000          0.000000          NA
##                                increment_in_explained
## CumErr + log_freq + stimlen          5.443207
## CumErr + log_freq          11.357233
## CumErr          83.199560
## null          0.000000
```

```
kable(deviance_differences_df[,2:3], format="latex", booktabs=TRUE) %>%
  kable_styling(latex_options="scale_down")
```

```
kable(deviance_differences_df[,c(4:6)], format="latex", booktabs=TRUE) %>%
  kable_styling(latex_options="scale_down")
```

```
NagContributions <- t(DAContributionAverage[3,])
NagTotal<-sum(NagContributions)
```

```

NagPercents <- NagContributions / NagTotal
NagResultTable <- data.frame(NagContributions = NagContributions, NagPercents = NagPercents)
names(NagResultTable)<-c("NagContributions","NagPercents")
write.csv(NagResultTable,paste0(TablesDir,CurPat,"_",RunLabel,"_",CurTask,
                                "_nagelkerke_contributions_table.csv"),row.names = TRUE)
NagPercents

```

```

##           Nagelkerke
## CumErr    0.6996656
## stimlen   0.1733182
## log_freq  0.1270162

```

```

sse_results_list<-compare_SS_accounted_for(FinalModelSet,"preserved ~ 1",PosDat,N_cutoff=10)

```

```

## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'CumErr'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'CumPres'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'CumErr'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'CumPres'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'CumErr'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'CumPres'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'CumErr'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'CumPres'. You can override using the `.groups` argument.

## Warning in (cumerr_residuals^2) * (N_values$cumerr_N): longer object length is not a multiple of shorter object length
## Warning in (null_cumerr_resid^2) * (N_values$len_pos_N): longer object length is not a multiple of shorter object length
## Warning in (null_cumcor_resid^2) * (N_values$len_pos_N): longer object length is not a multiple of shorter object length

## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'CumErr'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'CumPres'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'CumErr'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'CumPres'. You can override using the `.groups` argument.

## Warning in (cumerr_residuals^2) * (N_values$cumerr_N): longer object length is not a multiple of shorter object length

```

model	p_accounted_for	model_deviance
preserved ~ CumErr+log_freq+stimlen	0.8126290	3002.553
preserved ~ CumErr	0.8374256	3037.920
preserved ~ CumErr+log_freq	0.8390169	3014.012

```
## Warning in (null_cumerr_resid^2) * (N_values$len_pos_N): longer object length is not a multiple of shorter object length
## Warning in (null_cumcor_resid^2) * (N_values$len_pos_N): longer object length is not a multiple of shorter object length
## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'CumErr'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'CumPres'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'CumErr'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'CumPres'. You can override using the `.groups` argument.

## Warning in (cumerr_residuals^2) * (N_values$cumerr_N): longer object length is not a multiple of shorter object length
## Warning in (null_cumerr_resid^2) * (N_values$len_pos_N): longer object length is not a multiple of shorter object length
## Warning in (null_cumcor_resid^2) * (N_values$len_pos_N): longer object length is not a multiple of shorter object length
```

```
sse_table <- sse_results_table(sse_results_list)
write.csv(sse_table, paste0(TablesDir, CurPat, "_", RunLabel, "_",
                           CurTask, "_sse_results_table.csv"), row.names = TRUE)

sse_table
```

```
##               model p_accounted_for model_deviance diff_CumErr+log_freq+stimlen diff_CumErr diff_CumErr+log_freq
## 1 preserved ~ CumErr+log_freq+stimlen      0.8126290      3002.553              0.00000000 -0.024796587      -0.026387953
## 2                preserved ~ CumErr      0.8374256      3037.920              0.02479659  0.000000000      -0.001591367
## 3    preserved ~ CumErr+log_freq      0.8390169      3014.012              0.02638795  0.001591367      0.000000000
```

```
kable(sse_table[,1:3], format="latex", booktabs=TRUE) %>%
  kable_styling(latex_options="scale_down")
```

```
write.csv(results_report_DF, paste0(TablesDir, CurPat, "_", RunLabel, "_",
                                    CurTask, "_results_report_df.csv"), row.names = FALSE)
```

```
ncols_in_table <- ncol(sse_table)
kable(sse_table[,c(1,4:ncols_in_table)], format="latex", booktabs=TRUE) %>%
  kable_styling(latex_options="scale_down")
```

model	diff_CumErr+log_freq+stimlen	diff_CumErr	diff_CumErr+log_freq
preserved ~ CumErr+log_freq+stimlen	0.0000000	-0.0247966	-0.0263880
preserved ~ CumErr	0.0247966	0.0000000	-0.0015914
preserved ~ CumErr+log_freq	0.0263880	0.0015914	0.0000000